

Deep Learning for NLP

Student name: *Margarita Orfanidi*
ID: *7115152400023*

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

Contents

1	Abstract	3
2	Analysis	3
2.1	Dataset	3
2.2	Exploratory Data Analysis	4
2.2.1	Samples	4
2.2.2	Descriptive Analysis	5
2.2.3	Textual Analysis	11
2.2.4	Token Analysis	13
3	Pre-processing	14
3.1	Tokenization and Input Encoding	15
4	Model	16
4.1	BERT Variants	16
4.2	Fine-tuning Strategy	16
5	Evaluation	16
5.1	Cross-Validation	16
5.2	Evaluation Metrics and Diagnostic Tools	17
6	Experiments	17
6.0.1	Experiment 1: Baseline BERT Model	19
6.0.2	Experiment 2: Smaller Batch Size	21
6.0.3	Experiment 3: Increased Learning Rate	21
6.0.4	Experiment 4: Larger Batch Size	21
6.0.5	Experiment 5: Weighted Loss Function – Configuration [1.6, 1.0, 1.5]	21
6.0.6	Experiment 6: Weighted Loss Function – Configuration [1.4, 1.0, 1.5]	24
6.0.7	Experiment 7: Striding-Window Input Representation	25
6.0.8	Experiment 8: More Training Epochs	26
6.0.9	Experiment 9: More Training Epochs with Weighted Loss	27
6.0.10	Experiment 10: Weighted Loss with Warm-Up Scheduler	27
6.0.11	Overall Discussion	30
6.1	DistilBERT	30

6.1.1	Experiment 1: Baseline DistilBERT Model	30
6.1.2	Experiment 2: Increased Learning Rate	32
6.1.3	Experiment 3: Smaller Batch Size	33
6.1.4	Experiment 4: Smaller Batch Size with Increased Learning Rate .	33
6.1.5	Experiment 5: Larger Batch Size	35
6.1.6	Experiment 6: Weighted Cross-Entropy Loss	36
6.1.7	Experiment 7: Striding-Window Input Representation	36
6.1.8	Experiment 8: More Training Epochs	37
6.1.9	Experiment 9: More Epochs with Weighted Loss	37
6.1.10	Overall Discussion	38
6.2	DeBERTa Model	39
6.2.1	Experiment 1: Baseline DeBERTa Model	39
6.2.2	Experiment 2: Lower Learning Rate and Float32 Precision	41
6.2.3	Experiment 3: Weighted Loss and Warm-Up Scheduler	43
6.2.4	Experiment 4: Stronger Stabilization and Multi-Factor Tuning . .	44
6.2.5	Overall Discussion	47
6.3	Results	48
7	Conclusion and model selection	49
7.1	Comparison with the first project	50

1. Abstract

The identification of ambiguity in the responses politicians give during interviews is a highly challenging task in the field of NLP. The goal is to classify political responses into predefined categories that reflect the level of clarity or evasiveness present in an answer. Based on the dataset released for SemEval 2026 Task 6 (CLARITY), this research explores the problem by fine-tuning Transformer-based architectures, with a specific focus on the BERT model and its variations.

2. Analysis

2.1. Dataset

The dataset consists of a collection of questions and answers from Presidents of the United States during interviews. The dataset is divided into two parts: a training set and a test set. The training set contains 3,448 entries with 20 attributes, while the test set contains 308 entries with the same attributes. The columns are described in the following table.

Column	Description
title	Interview title
date	Date of the interview
president	President being interviewed
url	Source URL of the interview
question_order	Order of the question in the interview
interview_question	Original interview question
interview_answer	Answer given by the president
gpt3.5_summary	Summary generated by GPT-3.5
gpt3.5_prediction	Prediction generated by GPT-3.5
question	Sub question
annotator_id	Identifier of the annotator
annotator1	Label from annotator 1
annotator2	Label from annotator 2
annotator3	Label from annotator 3
inaudible	Indicates if the segment contains inaudible parts
multiple_questions	Indicates if the question contains multiple parts
affirmative_questions	Indicates if the question is affirmative
index	Entry index
clarity_label	Label indicating response clarity
evasion_label	Label indicating response evasion

Table 1: Columns of the SemEval 2026 Task 6 dataset.

The dataset includes textual components such as the original interview question, the sub-questions, and the corresponding interview response. Moreover, it contains metadata, such as the interview title, date, and the name of the president. Finally, the dataset includes the proposed classification labels provided at the sub-question level,

with each sub-question assigned classification labels (`clarity_label` and `evasion_label`) based on the proposed taxonomy.

It is important to note that the training set does not contain the individual labels assigned by each annotator, but only the final aggregated labels. Therefore, although the dataset generally contains non-null values, all values in these specific columns are null.

In contrast, in the test set, most metadata fields are null, while the columns necessary for evaluation are not null, such as the main text columns, the individual annotator labels (`annotator1`, `annotator2`, `annotator3`), and the corresponding labels.

2.2. Exploratory Data Analysis

Before building an accurate classifier, a study of the data and the nature of the problem must first be conducted. It is necessary to examine basic statistics, understand the distribution of the labels, and identify potential patterns and correlations in the data. Through this process, we can confirm the structure of the dataset, recognize the most important variables, and develop a clearer understanding of the problem we are trying to solve.

2.2.1. Samples. Random samples were displayed from the dataset in order to observe the structure and formatting of the textual data. This qualitative inspection helps identify potential patterns and understand how the text appears in practice, allowing us to draw conclusions about the preprocessing steps that may be required. For example, in the dataset, several spelling inconsistencies, missing spaces, and the presence of punctuation marks were observed, creating an initial intuition about the type of preprocessing that may be needed.

Sample 2900	
	Value
title	The President's News Conference With President Lee Myung-bak of South Korea at Camp David, Maryland
date	April 19, 2008
president	George W. Bush
url	https://www.presidency.ucsb.edu/documents/the-presidents-news-conference-with-president-lee-myung-bak-south-korea-camp-david
question_order	2
interview_question	Q. I have a question for President Lee. Korea and the United States have made many achievements through the summit meeting, especially North Korean nuclear issue and the strengthening of the alliance. As for North Korean nuclear issue, Mr. President Lee suggested setting up a permanent liaison office in both Seoul and Pyongyang. What are some of the follow-up effects, if you do have any follow-up actions? And do you have any thoughts of proposing a meeting with Chairman Kim at an earlier date?
interview_answer	The process is not something that we discussed between ourselves during the summit meeting. In fact, when I was staying in Washington, DC, I had an interview with one of the newspapers there, and it came up. Of course, it was not a sudden suggestion. I did have a meeting among my staff and related ministries, and I talked about this in detail before I came to the United States. We have a new administration in Korea, and we haven't yet to begun dialogue with the ns. Inter-Korean dialogue: there is a need for us to have dialogue all the time. Up until now, we had dialogues whenever the need arose, and then it would stop. However, dialogue should be based on genuine cooperation and sincerity. And so with this in mind, I thought that it would be helpful to set up a permanent liaison office in both Seoul and Pyongyang. As for the summit meeting between myself and Chairman Kim, I will agree to it when the need is real. And I already said publicly that I am willing to meet with him—not just once, but many times—but if the meeting will yield substantial and real results. I believe only when that is possible, I am ready to meet with him and have sincere dialogue, because that will help to bring about peace and stability of the peninsula. So basically, I do hold that thought, but I'm not suggesting that—to have a meeting with Chairman Kim anytime soon. The answer is more general in nature. Steven Lee [Steven Lee Myers, New York Times].
gpt3.5_summary	The question consists of 2 parts:\n1. Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.\n2. Thoughts on proposing a meeting with Chairman Kim at an earlier date.\n\n<- Summary Answer ->\n\nThe response provides the following information regarding these points:\n1. Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue:\n- President Lee suggests setting up a permanent liaison office in both Seoul and Pyongyang to facilitate dialogue and cooperation between North Korea and South Korea.\n- The suggestion was made during an interview with a newspaper in Washington, DC, and was not discussed during the summit meeting.\n- The intention behind the suggestion is to establish genuine cooperation and sincerity in inter-Korean dialogue.\n2. Thoughts on proposing a meeting with Chairman Kim at an earlier date:\n- President Lee is open to the idea of a meeting with Chairman Kim, but only when the need is real and there is a possibility of substantial and real results.\n- He believes that sincere dialogue with Chairman Kim can contribute to the peace and stability of the Korean peninsula.\n- While he holds the thought of meeting with Chairman Kim, he is not suggesting a meeting anytime soon, but is willing to meet if the need arises.\n\nOverall, President Lee emphasizes the importance of genuine cooperation, sincerity, and substantial results in both the setup of a liaison office and the possibility of a meeting with Chairman Kim.
gpt3.5_prediction	Question 1: Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.\nVerdict: 1.1 Explicit - The information requested is explicitly stated (in the requested form).\nExplanation: The response explicitly provides information about President Lee's suggestion to set up a permanent liaison office in Seoul and Pyongyang. It explains that the suggestion was made to facilitate dialogue and cooperation between North and South Korea and highlights the intention behind the suggestion.\nQuestion 2: Thoughts on proposing a meeting with Chairman Kim at an earlier date.\nVerdict: 2.4 General - The information provided is too general/lacks the requested specificity.\nExplanation: The response does not specifically address the question of proposing a meeting with Chairman Kim at an earlier date. While President Lee expresses willingness to meet with Chairman Kim when the need is real and there can be substantial results, he does not provide a direct response to the request for thoughts on proposing an earlier meeting. The answer is more general in nature.
question	Follow-up effects of setting up a permanent liaison office in Seoul and Pyongyang for the North Korean nuclear issue.
annotator_id	85
annotator1	None
annotator2	None
annotator3	None
inaudible	False
multiple_questions	False
affirmative_questions	True
index	2900
clarity_label	Ambivalent
evasion_label	Implicit

Figure 1: Sample

In addition, redundancy in the interview responses was observed. This occur because some responses are similar despite referring to different questions, but mainly because the responses are labeled based on whether they address each sub-question.

As a result, the same response may appear multiple times in the dataset, depending on the number of sub-questions associated with a particular interview question. This behavior is illustrated below by presenting the original question, the corresponding sub-questions, the interview response, and the assigned label. As shown, the same response is repeated across different sub-questions, while the assigned label may vary depending on the extent to which the response addresses each sub-question.

```
#####
INTERVIEW QUESTION:
Q. Mr. President, Turkey's ground offensive in northern Iraq is now a week old with no end in sight. How quickly would you like to see Turkey end its offensive, its incursion? And do you have any concerns about
#####
--- Example ---
Sub-question:
How quickly would you like to see Turkey end its offensive, its incursion?

Answer:
A couple of points on that-one, the Turks, the Americans, and the Iraqis, including the Iraqi Kurds, share a common enemy in the PKK. And secondly, it's in nobody's interests that there be safe haven for people.

Clarity label: Clear Reply

--- Example ---
Sub-question:
Do you have any concerns about the possibility of protracted presence in northern Iraq causing further destabilization in the region?

Answer:
A couple of points on that-one, the Turks, the Americans, and the Iraqis, including the Iraqi Kurds, share a common enemy in the PKK. And secondly, it's in nobody's interests that there be safe haven for people.

Clarity label: Ambivalent
```

Figure 2: Sample

This redundancy may make class separation more challenging, as very similar or identical textual responses can be associated with different labels depending on the specific sub-question being considered. Consequently, the model may find it difficult to distinguish between classes based solely on the response text, since the same linguistic content does not always correspond to the same label.

2.2.2. Descriptive Analysis.

Clarity Labels Distribution The labels that were used for the classification task are the clarity labels, which include Clear Reply, Clear Non-Reply, and Ambivalent. The distribution of instances across these labels is shown in Figure 3. The majority of instances belong to the Ambivalent class, accounting for approximately 59.2% of the data (2039 samples). The Clear Reply category represents about 30.5% of the dataset (1051 samples), while Clear Non-Reply is the least frequent class with 10.3% (356 samples). This distribution indicates a noticeable class imbalance, with the Ambivalent class dominating the dataset, a factor that should be considered during model training and evaluation.

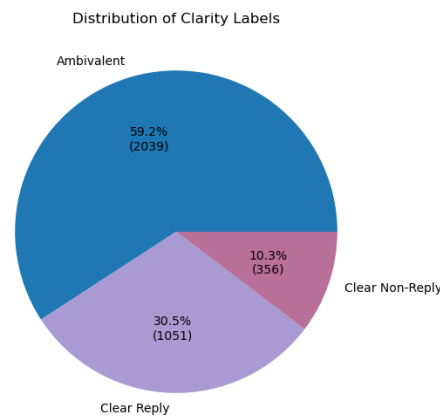


Figure 3: Distribution of clarity labels in the dataset

Evasion Labels Distribution The dataset also includes an `evasion_label`, which is a second, more detailed taxonomy that specifies the type of evasive strategy used in a response.

Figure 4 presents the relationship between clarity labels and evasion labels in the dataset. The evasion categories are Claims ignorance, Clarification, Declining to answer, Deflection, Dodging, Explicit, General, Implicit, and Partial/Half-answer. We can see that Deflection, Dodging, Implicit, General, and Partial/Half-answer correspond to Ambivalent responses, while Claims ignorance, Clarification, and Declining to answer correspond to Clear Non-Reply. The Explicit category corresponds to Clear Reply, meaning that the response is direct and clear.

evasion_label	Claims ignorance	Clarification	Declining to answer	Deflection	Dodging	Explicit	General	Implicit	Partial/half-answer
clarity_label									
Ambivalent	0	0	0	381	706	0	386	488	79
Clear Non-Reply	119	92	145	0	0	0	0	0	0
Clear Reply	0	0	0	0	0	1052	0	0	0

Figure 4: Relationship between clarity labels and evasion labels in the dataset.

Moreover, Figure 5 presents the distribution of the evasion labels in the dataset. The most frequent category is Explicit, accounting for 30.5% of the instances (1051 samples), which corresponds to clear and direct responses. Among the evasive strategies, Dodging appears most frequently (20.5%, 705 samples), followed by Implicit (14.2%, 487 samples), General (11.2%, 386 samples), and Deflection (11.0%, 380 samples). Less common categories include Declining to answer (4.2%), Claims ignorance (3.5%), Clarification (2.7%), and Partial/Half-answer (2.3%).

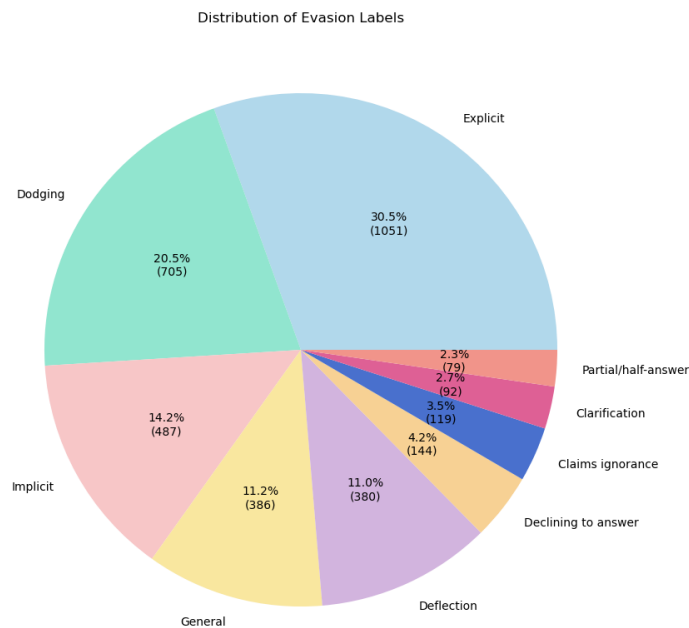
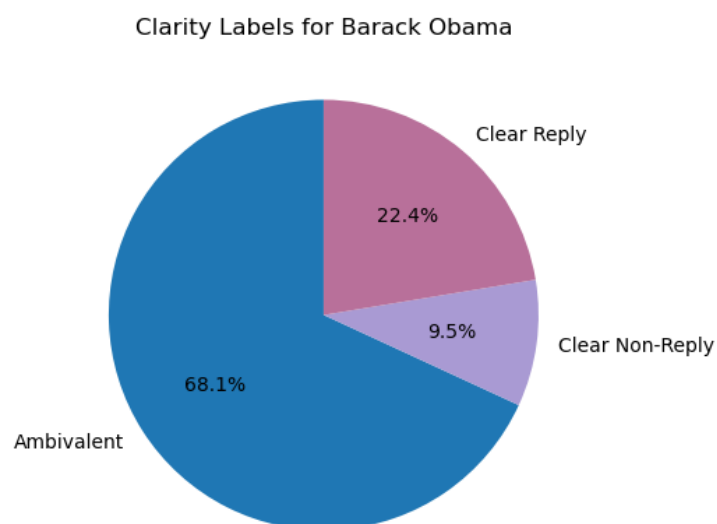
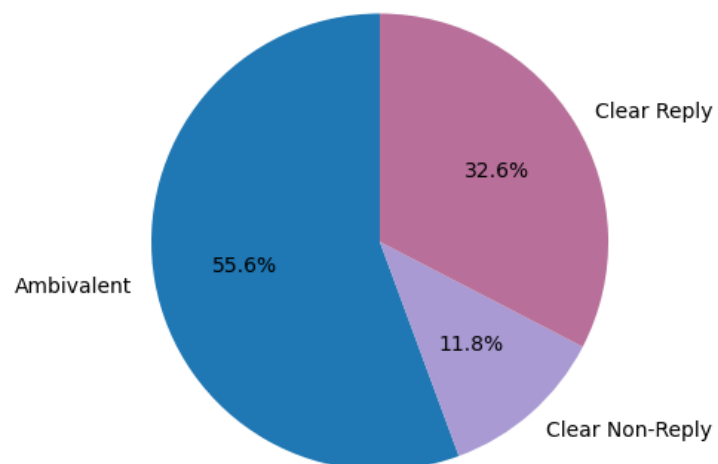


Figure 5: Distribution of evasion labels in the dataset.

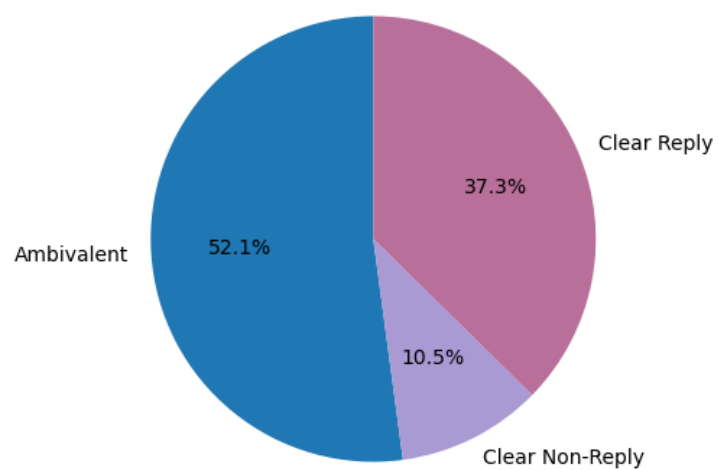
Clarity Label Distribution by President It is important to determine whether the nature of the responses in the dataset is speaker-dependent. An analysis of the distribution of clarity labels for each president helps reveal whether certain categories tend to appear more frequently depending on the president who provides the response, rather than representing a general characteristic of the dataset as a whole. As illustrated in the figures, Ambivalent responses constitute the majority for all presidents. However, some differences can still be observed. In particular, Barack Obama exhibits the highest proportion of Ambivalent responses (68.1%), indicating a larger share of indirect or partially evasive answers. In contrast, Donald J. Trump shows a higher proportion of Clear Reply responses (32.6%), suggesting that he provides more direct answers compared to the other presidents.

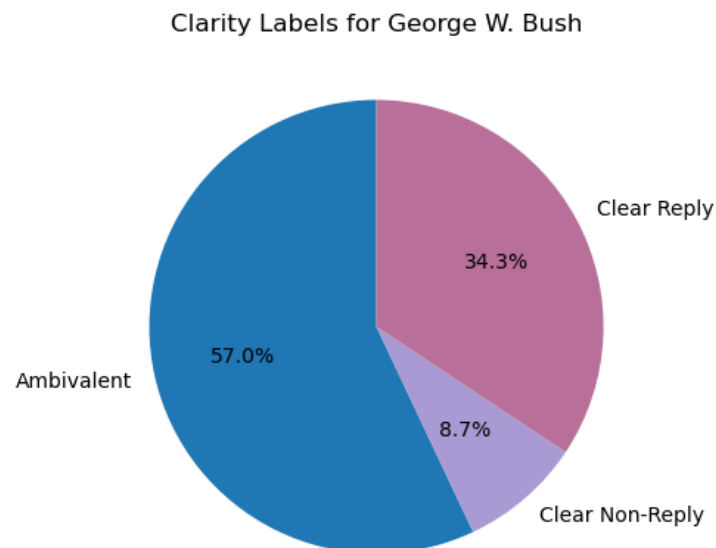


Clarity Labels for Donald J. Trump



Clarity Labels for Joseph R. Biden





To further compare these distributions, the label frequencies across presidents are visualized using a grouped bar chart. As shown in the figure, although minor differences can be observed, no substantial variations appear in the overall response patterns among the presidents. In all cases, Ambivalent responses remain the dominant category, followed by Clear Reply, while Clear Non-Reply appears less frequently. This suggests that the general response behavior is independent across the different presidents in the dataset.

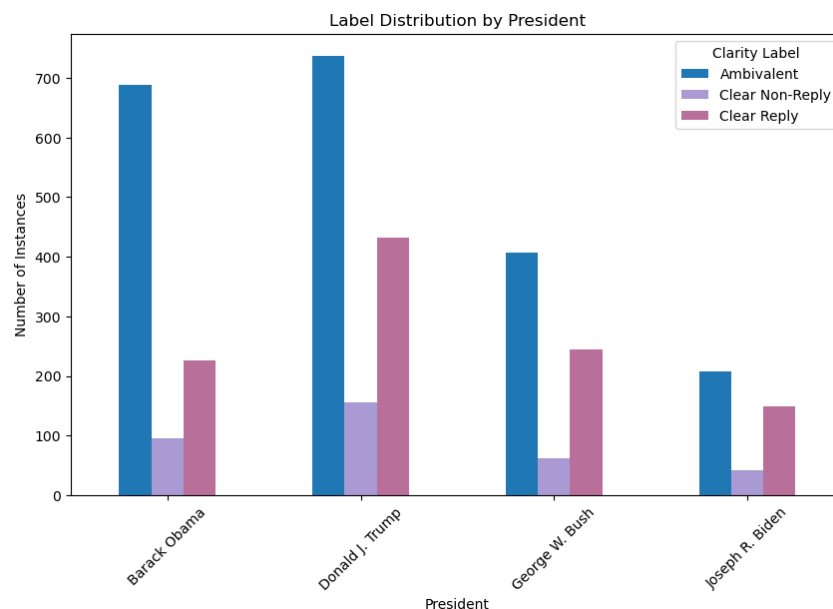


Figure 6: Label distribution per president

Clarity Labels Distribution by Affirmative questions Affirmative questions seem to be associated with slightly more direct responses. In these cases, Ambivalent responses become less frequent (from 60.5% to 54.5%), while Clear Reply (from 29.9% to 32.6%) and Clear Non-Reply responses (from 9.6% to 12.8%) become slightly more common.

This suggests that affirmative questions may encourage clearer answers, although the effect remains limited.

clarity_label	Ambivalent	Clear Non-Reply	Clear Reply
affirmative_questions			
False	0.605007	0.096039	0.298954
True	0.545337	0.128238	0.326425

Figure 7: Label distribution per affirmative questions

Clarity Labels Distribution by Year Figure 8 shows a heatmap of interview questions by president and year. The distribution of questions matches the years of each presidency, with questions concentrated in the relevant time periods. Some differences can also be seen in the number of questions per year, since some years contain more instances than others.

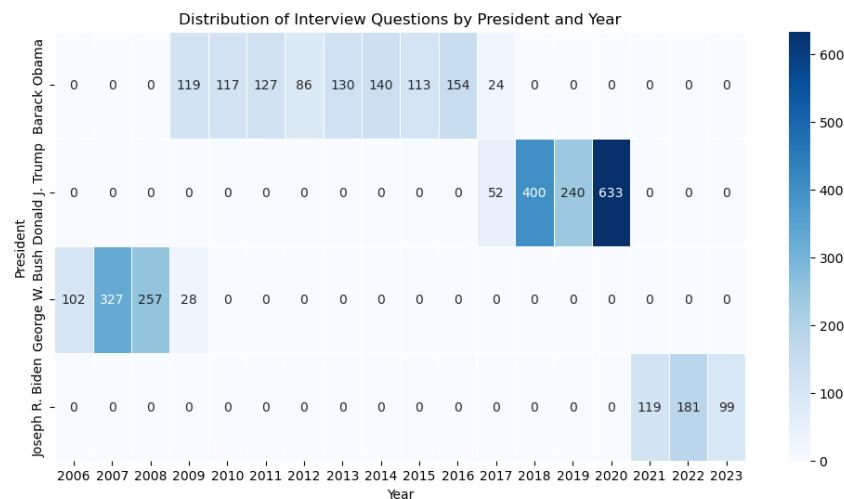


Figure 8: Distribution of interview answers per presidents and year

Figure 9 illustrates the distribution of clarity labels across different years. As observed, there is no significant temporal variation in the distribution, as the relative proportions of the labels remain stable over time.

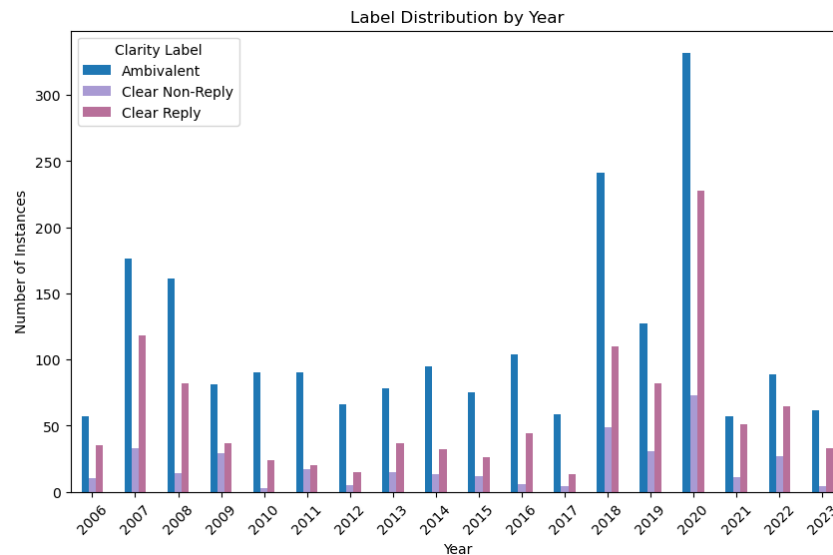


Figure 9: Distribution of clarity labels by year

2.2.3. Textual Analysis. The analysis of the dataset then focused on its textual characteristics. This type of analysis helps us better understand the linguistic properties of the responses, such as their length, vocabulary, and recurring word patterns. It also helps show whether certain textual features are related to specific clarity labels and whether the classes can be distinguished based on simple language patterns. In addition, this analysis can reveal possible challenges for the classification task, such as similar vocabulary across classes, the lack of clear lexical cues, or patterns that need preprocessing. These findings support the use of more informative text representations in the next stage of modeling.

Clarity Label Distribution by Word Count The analysis of answer length, measured in number of words, reveals notable differences across the clarity labels. The dataset contains 3,448 responses, with an average length of approximately 294 words and a high standard deviation (301.54), which indicates a variability in response length. The distribution is highly skewed, ranging from very short answers (1 word) to very long ones (up to 2,117 words). The median value is lower than the mean, suggesting the presence of long outlier responses that increase the average,.

When examining the classes separately, Ambivalent responses not only constitute the majority of the dataset but also tend to be longer on average (331.86 words). Clear Reply responses include 1,052 samples with a moderate average length of 272.04 words. In contrast, Clear Non-Reply responses form the smallest group and are considerably shorter, with an average length of 137.81 words.

Figure 10 illustrates these differences in answer length across the clarity labels. The boxplot shows that responses labeled as Ambivalent and Clear Reply tend to contain more words and exhibit greater variability in length compared to Clear Non-Reply. In contrast, Clear Non-Reply responses are generally shorter, with a lower median word count and a more concentrated distribution, although all classes contain several outliers.

Overall, these results suggest a clear relationship between response length and clarity labels. Shorter responses are more frequently associated with non-replies, while

longer and more variable responses tend to correspond to ambivalent answers. This observation suggests that response length may serve as a useful feature for distinguishing between clarity categories.

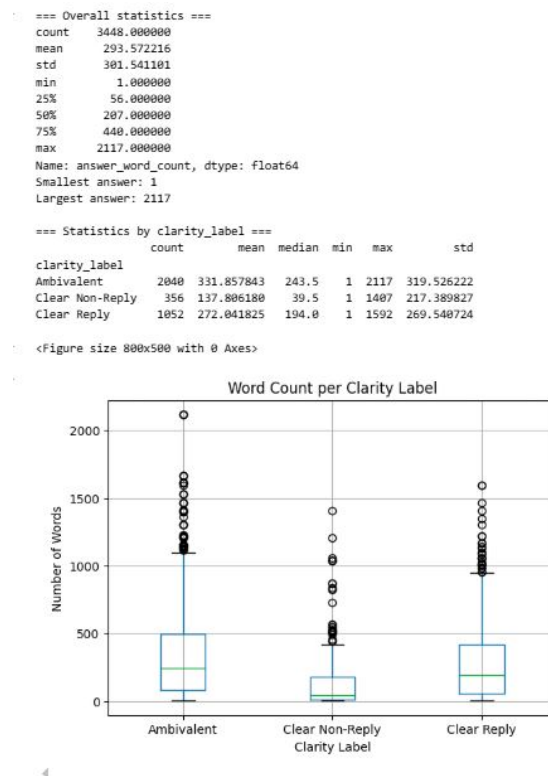


Figure 10: Words per label

Detection of text patterns in the dataset Although it is possible to manually inspect several examples from the dataset (e.g., by printing random entries), such inspection does not guarantee that all instances follow the same format. To systematically verify whether patterns are present in the dataset, a function named `detect_text_patterns` was implemented that scans the textual data and reports the occurrence of common types of noise. Specifically, the function searches for patterns such as HTML tags, email addresses, URLs, Twitter mentions, hashtags, digits, and newline characters using regular expressions. Additionally, it checks for the presence of emojis in the text. This automated inspection provides a quick overview of the dataset and helps determine whether specific cleaning operations are required before further processing. The results of this analysis indicated that the only notable pattern detected in the text column was the presence of numerical values.

```

detect_text_patterns(training_data["text"])
✓ 6.6s

Starting dataset inspection...
-----
Clean: No HTML Tags found.

Clean: No Emails found.

Clean: No URLs (http/www) found.

Clean: No Twitter Mentions (@) found.

Clean: No Hashtags (#) found.

Found: 1527 texts with Digits.
Clean: No Newlines (\n) found.

Clean: No Emojis found.

-----
Inspection completed!

```

Figure 11: Detection Patterns

Detection of spelling errors Moreover, a quality check was conducted to estimate the percentage of spelling errors present in the dataset. To do so, a function was implemented that scans the textual data and compares each word against an English dictionary using a spell-checking library. The function tokenizes the text into individual words, counts the total number of words, and identifies those that are not recognized by the dictionary as potential spelling errors. It then computes the percentage of misspelled words relative to the total number of words in the dataset. In addition, the function outputs a small set of example misspelled words to provide a qualitative indication of the types of spelling inconsistencies present in the data.

As shown in Figure 12, the analysis indicates that spelling errors constitute a very small portion of the dataset (approximately 1.96% of all words), suggesting that the textual data is generally clean and that spelling mistakes are unlikely to significantly affect the performance of the classification models. After removing stopwords, which usually carry limited semantic information for classification tasks, the percentage of misspelled words increased slightly to 2.94%. This suggests that spelling errors are somewhat more common in the more informative parts of the text, although their overall proportion remains relatively low.

```

check_spelling_errors(training_data["text"])
✓ 4.4s

Total words: 1125356
Misspelled words: 22040
Percentage of misspelled words: 1.96%
Example misspelled words: ['thei', 'chineseno', 'aboutit', 've', 'donei']

```

Figure 12: Spelling errors

2.2.4. Token Analysis. Since BERT has a strict architectural limitation of 512 tokens per sequence, it is critical to evaluate the token distribution produced by the BERT tokenizer on this dataset. The analysis shows a mean length of 398.10 tokens and a median

of 291.0. While the majority of samples fall within BERT's capacity, a significant 29.41% of the dataset exceeds the 512-token limit. Because nearly one-third of the sequences are subject to truncation, there is a substantial risk of losing key information.

```
Total samples: 3448
Mean length: 397.24
Median length: 291.0
Maximum length: 2595
Minimum length: 7

Samples exceeding 512 tokens: 1014 (29.41%)
```

Figure 13: Tokens

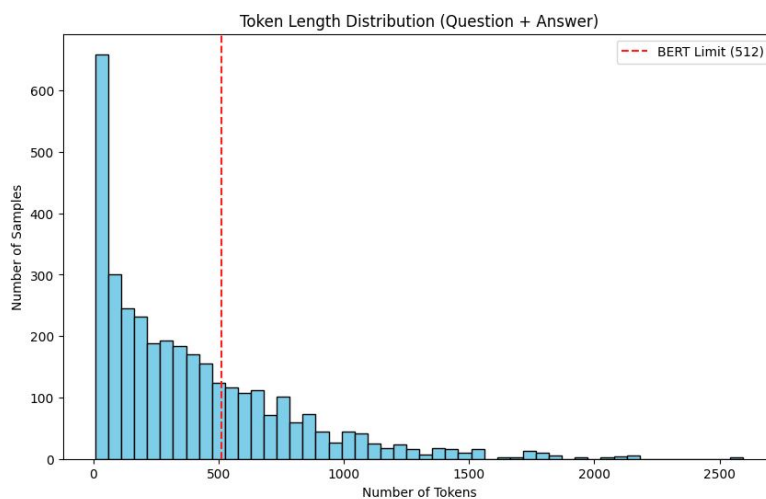


Figure 14: Tokens distribution

3. Pre-processing

Based on the findings from the exploratory data analysis, a simple and targeted pre-processing pipeline was adopted. The initial detection of text patterns revealed that the dataset was relatively clean, with no significant presence of noise such as HTML tags, URLs, hashtags, or Twitter mentions. Furthermore, the spelling error analysis indicated that misspelled words accounted for less than 2% of the total vocabulary. Given that Transformer-based models such as BERT rely on subword tokenization, which is inherently robust to minor spelling variations and numerical values, no aggressive cleaning or spelling correction was deemed necessary. Similarly, stopwords removal was avoided, since common function words, including prepositions, pronouns, and auxiliary verbs, can contribute to the syntactic and semantic structure of the input. In Transformer-based models, such words may influence the contextual representations learned by the model; therefore, removing them could result in the loss of useful information for capturing the relationship between the question and the answer.

The following specific steps were applied:

1. Type Casting: The question and interview_answer columns in both the training and test sets were explicitly cast to string type to ensure compatibility with the tokenizer and prevent execution errors.

2. Deduplication: Any duplicate rows—defined by identical combinations of the question, interview_answer, and clarity_label—were removed from the training set. This step ensures that the model does not overfit to repeated question-answer triplets and maintains the integrity of the learning process.
3. Input Formatting: Since the clarity of a response is inherently linked to the specific question asked, the question and interview_answer fields were provided to the tokenizer as paired inputs, allowing the model to capture the relationship between the two text segments.

```
Before drop_duplicates: (3448, 20)
After drop_duplicates: (3402, 20)
```

Figure 15: Number of training rows after duplicate removal

3.1. Tokenization and Input Encoding

For the tokenization process, the tokenizer corresponding to each pretrained checkpoint was used. Since three model variations were evaluated in this study, the tokenizer was initialized separately for each variation to ensure compatibility with the respective model architecture and vocabulary. For example, for the bert-base-uncased checkpoint, the tokenizer was initialized as follows:

```
model_checkpoint = "bert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
```

The same tokenization procedure was applied across all model variations. The tokenizer was applied to each question and its corresponding interview answer as a paired input. Specifically, the two text fields were provided as two separate lists:

```
train_encodings = tokenizer(
    list(df_train_fold["question"]),
    list(df_train_fold["interview_answer"]),
    truncation=True,
    padding=True,
    max_length=512
)
```

This allowed each Transformer-based model to encode every question-answer pair into a single input sequence using the standard paired-sequence format:

[CLS] question [SEP] interview answer [SEP]

In this format, the [CLS] token is placed at the beginning of the sequence and is typically used for classification tasks, while the [SEP] tokens separate the question from the interview answer and mark the end of the sequence.

To ensure consistent input dimensions and efficient batch processing, both truncation and padding were enabled. Truncation was necessary because several question-answer pairs exceeded the maximum input length supported by the models. The maximum sequence length was set to 512 tokens, which is the maximum capacity supported by the BERT-based models used in this study, allowing the models to retain as

much contextual information as possible from the interview transcripts. Padding was applied so that all sequences in a batch had the same length, enabling efficient tensor operations during training.

4. Model

In this work, several Transformer-based models were fine-tuned for a three-class sequence classification task. The objective was to predict one of the three clarity labels for each question–answer pair. For all experiments, `AutoModelForSequenceClassification` was used with the corresponding pretrained checkpoint and three output labels.

4.1. BERT Variants

BERT is an encoder-only model based on the Transformer architecture. Over time, the original BERT architecture has been adapted and extended in several ways to improve its performance and efficiency. In this assignment, three variants are examined:

- **BERT:** The original BERT model is pretrained using Masked Language Modeling (MLM), where the model learns to predict masked tokens in the text, and Next Sentence Prediction (NSP), where the model predicts whether one text passage is likely to follow another.
- **DistilBERT:** DistilBERT is a distilled and lighter version of BERT. It retains approximately 97% of BERT’s language understanding performance while being 60% faster and 40% smaller.
- **DeBERTa:** DeBERTa introduces two main architectural changes. First, it uses disentangled attention, where each token is represented using separate vectors for content and relative position, allowing the self-attention layers to better model dependencies between tokens. Second, it incorporates an enhanced mask decoder, where absolute position information is used during token decoding, since absolute word position can also be important for prediction.

4.2. Fine-tuning Strategy

The models were trained using the AdamW optimizer. Several training configurations were tested in order to evaluate the effect of different hyperparameters, including the learning rate, batch size, number of epochs, loss function, and input-handling strategy.

Standard cross-entropy loss was used in the baseline experiments. In later experiments, class-weighted cross-entropy loss was also tested to address class imbalance. Linear learning rate schedulers and warm-up steps were introduced in selected configurations to support smoother optimization and improve training stability.

5. Evaluation

5.1. Cross-Validation

To ensure the generalizability and stability of the results, stratified cross-validation was implemented. Stratification was used to preserve approximately the same label distribution in each fold as in the original dataset, which is particularly important due to the imbalanced nature of the data.

For BERT and DistilBERT, most experiments used 4-fold cross-validation. For DeBERTa, selected experiments used 2-fold cross-validation due to its higher computational cost and the need for additional stabilization tests.

In each fold, the model was trained on the training subset and evaluated on the corresponding validation subset. The best epoch for each fold was selected based on the highest validation Macro F1-score. The predictions and labels from these best-performing epochs were stored and later aggregated as out-of-fold predictions, providing a more reliable overall evaluation across the full training set.

5.2. Evaluation Metrics and Diagnostic Tools

Given the class imbalance in the dataset, the Macro-averaged F1-score was used as the primary evaluation metric. Unlike accuracy, which can be biased toward the majority class, Macro F1 assigns equal importance to each class. Therefore, it provides a more informative assessment of the model's ability to recognize both majority and minority classes.

To further analyze the model's behavior and training stability, the following diagnostic tools were used:

- **Out-of-Fold (OOF) Analysis:** Predictions from the validation subset of each fold were aggregated to produce a global classification report. This allowed us to evaluate the model's performance across the entire training set, rather than relying on a single validation split.
- **Classification Report:** A classification report was generated using the aggregated out-of-fold predictions. This report includes accuracy, precision, recall, and F1-score for each class, as well as macro and weighted averages. It provides a more detailed evaluation of the model's performance per class and helps identify whether the model performs worse on minority classes.
- **Normalized Confusion Matrix:** A normalized confusion matrix was generated to examine the distribution of predictions across the three classes. This helped identify systematic misclassification patterns, such as whether the model frequently confused specific clarity levels.
- **Learning Curves:** Training and validation loss, together with the Macro F1-score, were plotted across epochs. These curves were used to monitor the learning process, detect overfitting, and assess whether the model converged consistently across different folds.

6. Experiments

The first experiment defines the baseline training setup for the three-class classification task. This configuration was used as the reference point for all experiments.

The baseline configuration consisted of 4 folds, 3 training epochs, a batch size of 8, and a learning rate of 2×10^{-5} . The model was optimized using the AdamW optimizer and trained with standard cross-entropy loss. A linear learning rate scheduler was applied without warm-up steps. The primary evaluation metric was the Macro F1-score.

For each training example, the model received two textual inputs: the question and the corresponding interview answer. These two fields were encoded as a paired sequence using the BERT tokenizer. The resulting input followed the standard BERT format:

[CLS] question [SEP] interview answer [SEP]

The tokenizer was applied with truncation, padding, and a maximum sequence length of 512 tokens. Truncation was enabled because, as shown in the token analysis, several input sequences exceeded BERT's maximum token limit. Padding was applied in order to create batches with uniform sequence lengths. The maximum length of 512 tokens was selected because it corresponds to the maximum input length supported by BERT, allowing the model to retain as much contextual information as possible from the interview transcripts while remaining compatible with the model architecture.

A stratified K-fold cross-validation strategy was used with 4 folds. Stratification was important because the dataset is imbalanced, and it ensured that each fold maintained approximately the same label distribution as the full training dataset. For each fold, the training and validation subsets were created, the question-answer pairs were tokenized, custom PyTorch datasets were constructed, and DataLoaders were initialized for batch-based training and evaluation. The training DataLoader used shuffling, while the validation DataLoader did not, since the validation set was only used for evaluation.

For every fold, a new instance of `AutoModelForSequenceClassification` was initialized from the selected pretrained checkpoint and configured with `num_labels=3`, corresponding to the three target classes of the classification task. When multiple GPUs were available, the model was wrapped using `torch.nn.DataParallel` to support parallel computation. The model was then moved to the selected device. Re-initializing the model at the beginning of each fold ensured that every fold was trained independently and prevented information leakage between folds.

During training, the model was set to training mode. For each batch, the input IDs, attention masks, and labels were moved to the selected device. A forward pass was performed, the loss was computed, backpropagation was applied, the optimizer updated the model parameters, and the learning rate scheduler was stepped. Predictions and labels were stored in order to compute the training Macro F1-score. At the end of each epoch, the average training loss and training Macro F1-score were calculated.

After each training epoch, the model was evaluated on the validation set. The model was switched to evaluation mode, and gradient computation was disabled using `torch.no_grad()`. For every validation batch, a forward pass was performed, the validation loss was computed, and the predicted class was obtained from the highest logit value. The validation performance was measured using the average validation loss and the validation Macro F1-score.

For each fold, the best validation Macro F1-score was tracked across epochs. Instead of relying only on the final epoch, the predictions and labels corresponding to the epoch with the highest validation Macro F1-score were kept. This allowed the final

cross-validation results to be based on the best-performing epoch of each fold, rather than on the last epoch, which could already be affected by overfitting. The best predictions and labels from each fold were also collected as out-of-fold predictions, which were later used to produce the overall classification report and normalized confusion matrix.

At the end of each fold, the model and optimizer were deleted, and the CUDA cache was cleared in order to manage GPU memory efficiently and ensure that each fold started from a clean computational state.

After all folds were completed, the overall cross-validation performance was reported. The final evaluation included the mean validation Macro F1-score across folds, the standard deviation of the validation Macro F1-score, the overall classification report based on out-of-fold predictions, and the normalized confusion matrix. The mean and standard deviation provided an estimate of the model's average performance and stability across different validation splits, while the classification report and normalized confusion matrix offered a more detailed view of per-class behavior and misclassification patterns.

6.0.1. Experiment 1: Baseline BERT Model. In the first experiment, a baseline BERT model was trained using a standard fine-tuning configuration. The training was performed for 3 epochs, with a batch size of 8 and a learning rate of 2×10^{-5} .

```
=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.6009 ± 0.0161

Fold execution times:
Fold 1: 9.66 minutes
Fold 2: 9.71 minutes
Fold 3: 9.71 minutes
Fold 4: 9.72 minutes

Total time for all 4 folds: 38.80 minutes
Average time per fold: 9.70 minutes

Overall Classification Report (Out-Of-Fold Predictions):
```

	precision	recall	f1-score	support
0	0.59	0.41	0.49	1041
1	0.69	0.80	0.74	2009
2	0.59	0.57	0.58	352
accuracy			0.66	3402
macro avg	0.62	0.59	0.60	3402
weighted avg	0.65	0.66	0.65	3402

Figure 16:

As shown in Figure 19, the baseline model achieved a mean Macro F1-score of

0.6009 with a standard deviation of 0.0161 across the four folds. The weighted F1-score was 0.65, indicating that the model achieved reasonable overall performance. The relatively low standard deviation also suggests that the baseline configuration was stable across the different validation splits.

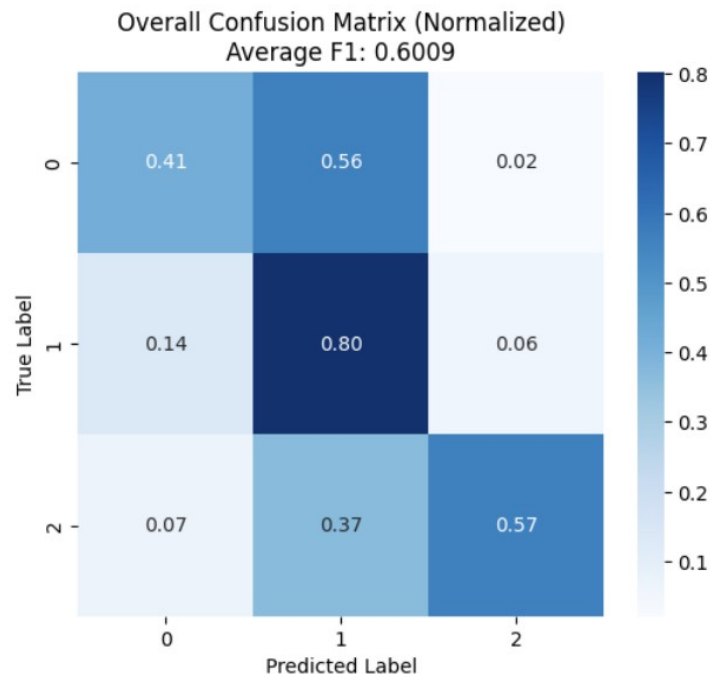


Figure 17:

The confusion matrix in Figure 17 shows that the majority class is classified more reliably, while the minority classes show lower recall and are often confused with the dominant Ambivalent class. This is also evident from the low recall values of these classes, as 56% of the samples from one minority class and 37% of the samples from the other minority class are misclassified. This pattern suggests that the model is affected by the class distribution of the dataset and motivates the use of class weighting in later experiments.

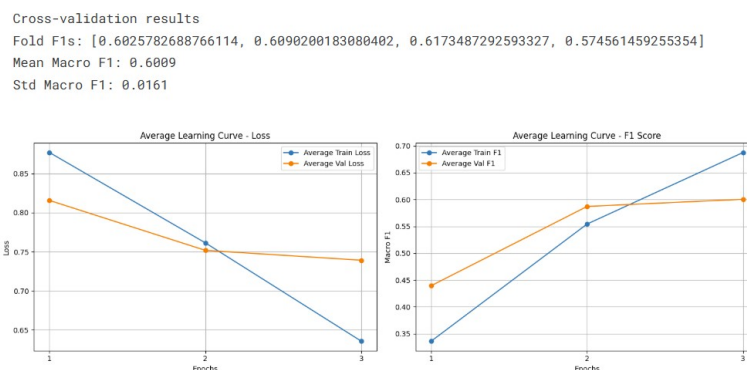


Figure 18: .

As shown in Figure 18, the loss curves show a smooth and consistent decrease in training loss across the three epochs. The validation loss follows a similar trend and remains close to the training loss. This indicates that the model did not show strong signs of overfitting within the 3-epoch training budget, suggesting that this duration was appropriate for the baseline setup.

Overall, this experiment established a strong reference point for the following BERT experiments. The standard fine-tuning configuration was able to achieve stable performance without clear signs of overfitting. However, the confusion matrix revealed that the model was still affected by the class imbalance of the dataset, as the minority classes were more frequently misclassified. Therefore, this baseline motivated the exploration of alternative configurations, such as class-weighted loss, different learning rates, batch sizes, additional epochs, and learning-rate scheduling.

6.0.2. Experiment 2: Smaller Batch Size. In the second experiment, we investigated the effect of reducing the batch size. The batch size was reduced from 8 to 4, while the learning rate remained 2×10^{-5} and the number of epochs remained 3. The purpose of this experiment was to examine whether smaller batches and more frequent parameter updates could improve generalization.

The model achieved a mean macro F1-score of 0.5958 with a standard deviation of 0.0167. Although this result was close to the baseline, it did not improve performance. The weighted F1-score remained approximately 0.65. This suggests that reducing the batch size alone did not provide a meaningful benefit for this BERT configuration.

6.0.3. Experiment 3: Increased Learning Rate. In the third experiment, we tested a higher learning rate. The learning rate was increased from 2×10^{-5} to 3×10^{-5} , while the batch size remained 8 and the model was trained for 3 epochs. The goal was to examine whether a more aggressive update step could help the model converge faster or reach a better solution.

The model achieved a mean macro F1-score of 0.5869 with a standard deviation of 0.0341. The validation accuracy was approximately 0.66 and the weighted F1-score was 0.64. Compared to the baseline, the macro F1-score decreased and the standard deviation increased. This indicates that the higher learning rate made training less stable across folds and did not improve overall performance.

6.0.4. Experiment 4: Larger Batch Size. In the fourth experiment, we evaluated the effect of increasing the batch size. The batch size was increased to 16, while the learning rate was kept at 3×10^{-5} and the number of epochs remained 3. This experiment was designed to test whether larger batches could produce more stable gradient estimates and improve training efficiency.

The model achieved a mean Macro F1-score of 0.5815 with a standard deviation of 0.0272, while the weighted F1-score was reduced to 0.63. This result was lower than the baseline showing that the larger batch size did not improve classification performance.

6.0.5. Experiment 5: Weighted Loss Function – Configuration [1.6, 1.0, 1.5]. In the fifth experiment, we introduced a weighted cross-entropy loss function in order to address class imbalance. Since the dataset contains classes with different frequencies, class

weights were applied so that the model would give more importance to the underrepresented classes during training. The model was trained for 3 epochs, using a batch size of 8 and a learning rate of 2×10^{-5} . The class weights were set to [1.6, 1.0, 1.5].

```
=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.5940 ± 0.0069

Fold execution times:
Fold 1: 9.20 minutes
Fold 2: 9.20 minutes
Fold 3: 9.20 minutes
Fold 4: 9.21 minutes

Total time for all 4 folds: 36.81 minutes
Average time per fold: 9.20 minutes

Overall Classification Report (Out-Of-Fold Predictions)
      precision    recall  f1-score   support

0         0.55      0.40      0.46      1041
1         0.69      0.77      0.73      2009
2         0.56      0.63      0.59       352

accuracy          0.64      3402
macro avg         0.60      0.60      0.59      3402
weighted avg      0.63      0.64      0.63      3402
```

Figure 19:

This setup achieved a mean Macro F1-score of 0.5940 with a standard deviation of 0.0069. Although it did not outperform the baseline, it produced more stable results across folds, as indicated by the smaller standard deviation. In addition, the classification report showed improved recall for the minority classes, suggesting that the weighted loss helped the model recognize underrepresented classes more effectively. However, this improvement came with a slight trade-off, as the recall of the majority class decreased from 80% in the baseline experiment to 77%. This indicates that the model became more sensitive to the underrepresented classes, but slightly less accurate in identifying the dominant class.

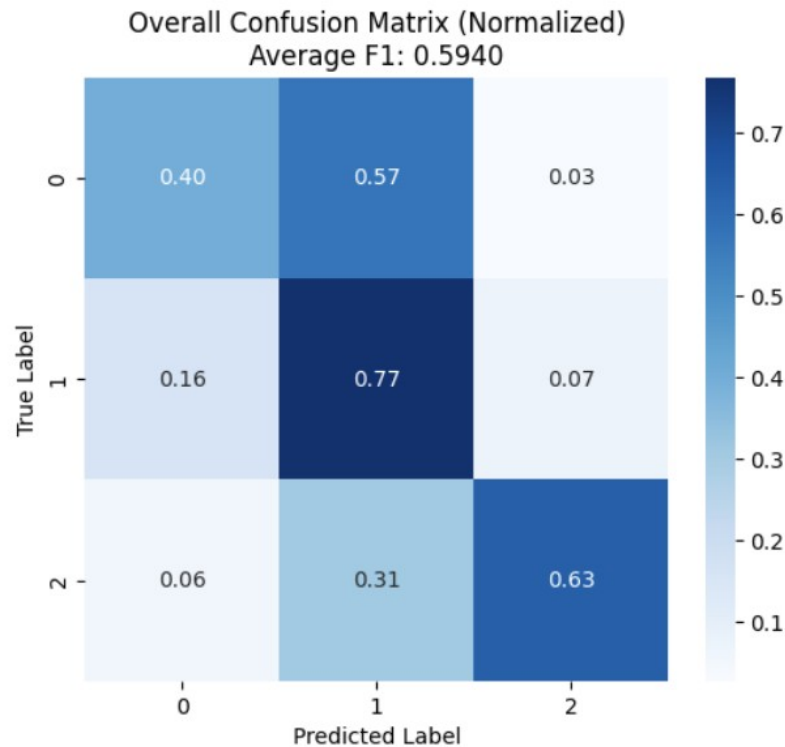


Figure 20:

Compared to the baseline confusion matrix, this configuration helped the model distinguish the Clear Non-Reply class more effectively, reducing its misclassified samples from 37% to 31%. However, this improvement came with a slight trade-off, as the model confused the Ambivalent class more often by classifying some of its samples as Clear Reply. This suggests that the weighted loss improved the recognition of one minority class, but also slightly shifted the decision boundary between Ambivalent and Clear Reply responses.

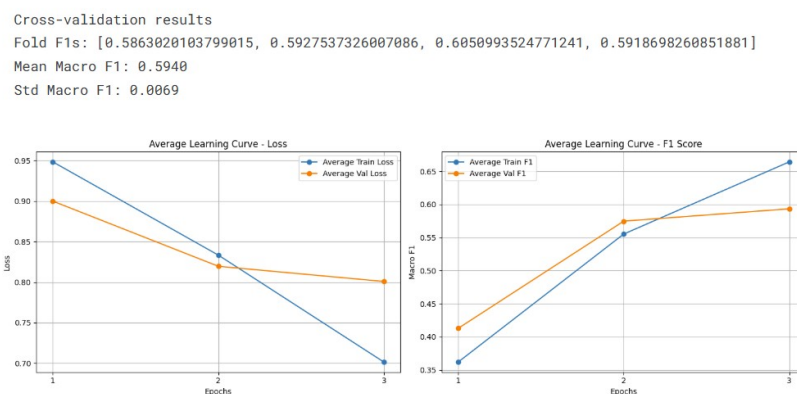


Figure 21:

The loss curves remain stable, with training and validation loss decreasing consistently across epochs. No strong signs of overfitting are observed. The low standard

deviation of 0.0069 across folds also indicates that this weighted-loss configuration produced stable and reproducible results.

6.0.6. Experiment 6: Weighted Loss Function – Configuration [1.4, 1.0, 1.5]. In the sixth experiment, we tested a second weighted-loss configuration using class weights [1.4, 1.0, 1.5]. The remaining hyperparameters were kept identical to Experiment 5.

```
=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.5964 ± 0.0080

Fold execution times:
Fold 1: 9.25 minutes
Fold 2: 9.25 minutes
Fold 3: 9.24 minutes
Fold 4: 9.27 minutes

Total time for all 4 folds: 37.02 minutes
Average time per fold: 9.25 minutes

Overall Classification Report (Out-Of-Fold Predictions)
      precision    recall  f1-score   support

0         0.55        0.41        0.47        1041
1         0.69        0.78        0.73        2009
2         0.59        0.58        0.58         352

 accuracy         0.65        3402
 macro avg        0.61        0.59        0.60        3402
 weighted avg     0.64        0.65        0.64        3402
```

Figure 22:

This configuration achieved a mean Macro F1-score of 0.5964 with a standard deviation of 0.0080, while the weighted F1-score reached 0.64. This result was better than the first weighted-loss configuration, but it was still slightly below the baseline Macro F1-score. However, the classification report showed an improvement in the recall of the minority classes compared to both the baseline and the previous weighted-loss experiment. Specifically, the recall increased to 41% for Clear Reply and 63% for Clear Non-Reply. This improvement came with only a small decrease in the recall of the majority class, which dropped from 80% in the baseline experiment to 78%. Overall, this configuration provided a better balance between minority-class recognition and majority-class performance, although it did not achieve the highest overall Macro F1-score.

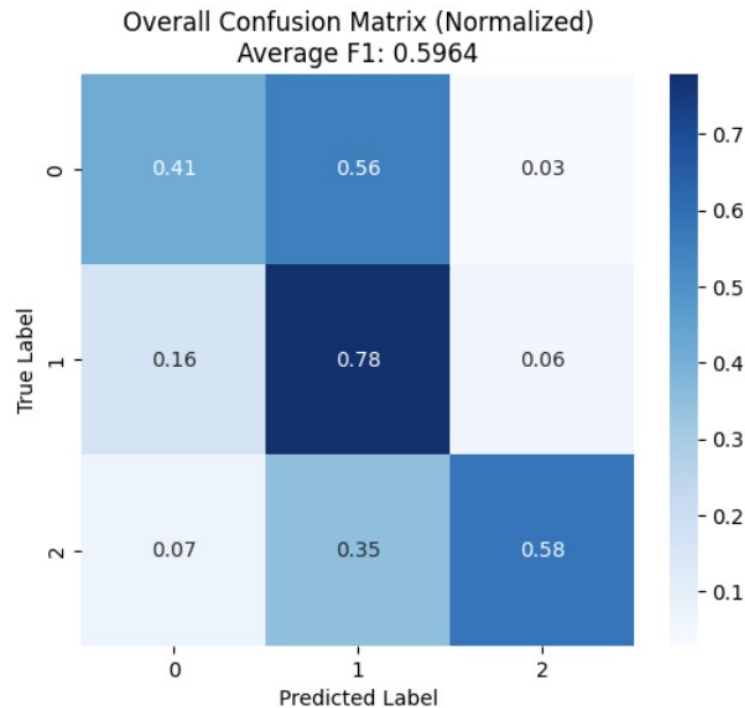


Figure 23:

Reducing the weight for class 0 from 1.6 to 1.4 produced a slightly better trade-off between sensitivity to the minority classes and overall prediction accuracy. This is reflected in the marginally higher macro F1-score of 0.5964. However, the improvement was still not sufficient to outperform the baseline.

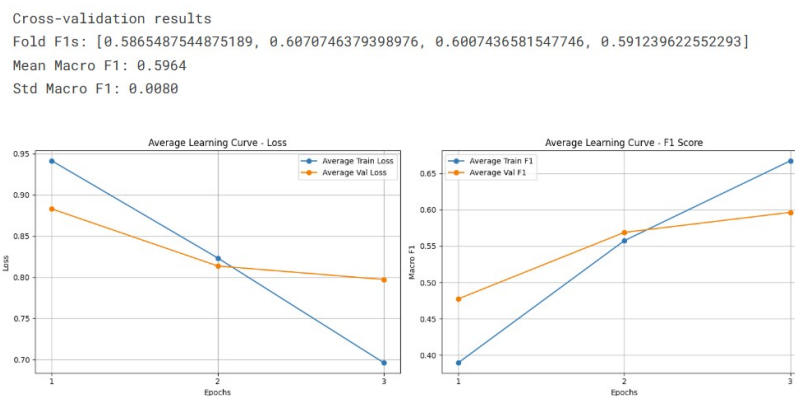


Figure 24:

The training dynamics are similar to those observed in Experiment 5. Both training and validation loss decrease smoothly, and the gap between them remains small, indicating good generalization within the 3-epoch training window.

6.0.7. Experiment 7: Striding-Window Input Representation. In the seventh experiment, the input representation was modified by applying a striding-window approach.

This technique was tested because the token analysis showed that approximately 30% of the question–answer pairs exceeded BERT’s maximum input length of 512 tokens. Therefore, the goal was to better handle longer inputs and reduce the amount of potentially useful information lost due to truncation.

The maximum sequence length was set to 512 tokens, and a stride of 128 tokens was used. The model was trained for 3 epochs, with a batch size of 8 and a learning rate of 2×10^{-5} .

The model achieved a mean Macro F1-score of 0.5835 with a standard deviation of 0.0169. Despite the more advanced input-processing strategy, the model did not improve over the baseline. This suggests that, for this dataset, the additional context captured by the striding-window approach was not sufficient to improve classification performance. It is also possible that the extra input segments introduced additional noise or made the prediction process more complex.

6.0.8. Experiment 8: More Training Epochs. In the eighth experiment, we increased the number of training epochs from 3 to 4. The batch size remained 8 and the learning rate was set to 2×10^{-5} . The goal of this experiment was to examine whether allowing the model to train for longer would improve performance.

The model achieved a mean macro F1-score of 0.5983 with a standard deviation of 0.0216. This was one of the best-performing non-weighted configurations and was very close to the baseline result. However, it did not surpass the baseline macro F1-score.

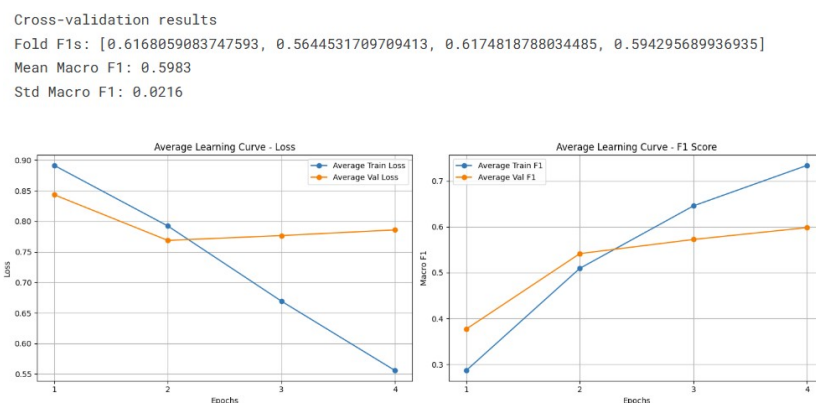


Figure 25:

The loss curves show that the model continued to reduce the training loss during the fourth epoch. However, the validation loss began to stagnate or slightly increase. This suggests early signs of overfitting, where the model starts to fit training-specific patterns without improving generalization. This behavior is consistent with the fact that the macro F1-score did not improve compared to the 3-epoch baseline. From the Macro F1 learning curve, it can be observed that after the third epoch the validation Macro F1-score reaches a plateau, with only a very small improvement afterwards. Therefore, the additional training time is not justified, especially given the increased risk of overfitting.

6.0.9. Experiment 9: More Training Epochs with Weighted Loss. In the ninth experiment, we combined 4 training epochs with class weights [1.3, 1.0, 1.5]. The model was trained using the same batch size and learning rate as the baseline. This setup was tested to examine whether weighted loss would benefit from one additional epoch of training.

This configuration achieved a mean macro F1-score of 0.5981 with a standard deviation of 0.0063. Although it was very stable across folds, it did not improve over the best 3-epoch weighted-loss setup. This suggests that training for one additional epoch did not provide additional generalization benefits and may have started to overfit the training data.

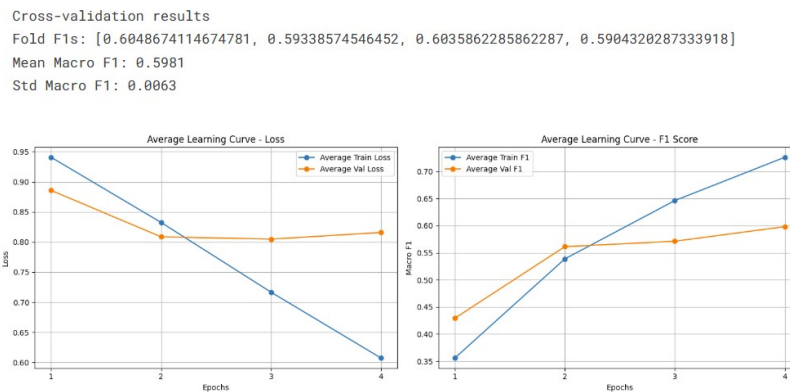


Figure 26:

Similarly to Experiment 8, the loss curves show a growing divergence between training and validation loss during the fourth epoch. Despite the very low standard deviation of 0.0063, the macro F1-score did not improve beyond the best 3-epoch weighted configurations. This further supports the conclusion that 3 epochs are more appropriate for this model and dataset combination. From the Macro F1 learning curve, it can be observed that after the third epoch the validation Macro F1-score reaches a plateau, with only a very small improvement afterwards. Therefore, the additional training time is not justified, especially given the increased risk of overfitting.

6.0.10. Experiment 10: Weighted Loss with Warm-Up Scheduler. In the tenth experiment, we combined class weights [1.3, 1.0, 1.5] with a warm-up learning rate scheduler. The model was trained for 3 epochs using a batch size of 8 and a base learning rate of 2×10^{-5} . The warm-up steps were included to allow the model to gradually increase its learning rate at the beginning of training, which can help stabilize optimization when class weighting is applied.

```

=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.6041 ± 0.0162

Fold execution times:
Fold 1: 8.91 minutes
Fold 2: 8.90 minutes
Fold 3: 8.89 minutes
Fold 4: 8.90 minutes

Total time for all 4 folds: 35.62 minutes
Average time per fold: 8.90 minutes

Overall Classification Report (Out-Of-Fold Predictions):
      precision    recall  f1-score   support

0         0.55      0.45      0.49       1041
1         0.69      0.77      0.73       2009
2         0.63      0.56      0.59        352

 accuracy          0.65       3402
  macro avg         0.62      0.59      0.61       3402
 weighted avg         0.64      0.65      0.64       3402

```

Figure 27:

This configuration achieved the highest mean macro F1-score among all tested BERT experiments, reaching 0.6041 with a standard deviation of 0.0162. The combination of class weighting and the warm-up scheduler improved the balance between the classes and produced the best overall BERT performance.

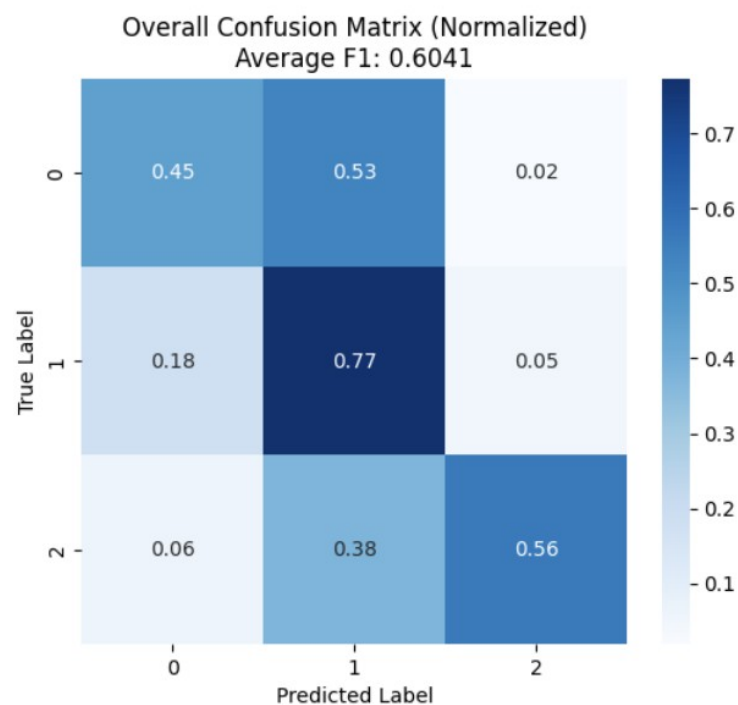


Figure 28:

The confusion matrix and the classification report indicate that this experiment achieved the best trade-off in performance across the three classes. Compared to the previous weighted-loss configurations, the model produced a more balanced distribution of predictions while maintaining strong performance on the majority class. In particular, the recall of the Clear Reply class increased to 45%, the recall of the majority Ambivalent class remained stable at 77%, and the Clear Non-Reply class achieved a recall of 56%. This balance between improving minority-class recognition and preserving majority-class performance is reflected in the highest Macro F1-score among the BERT experiments.

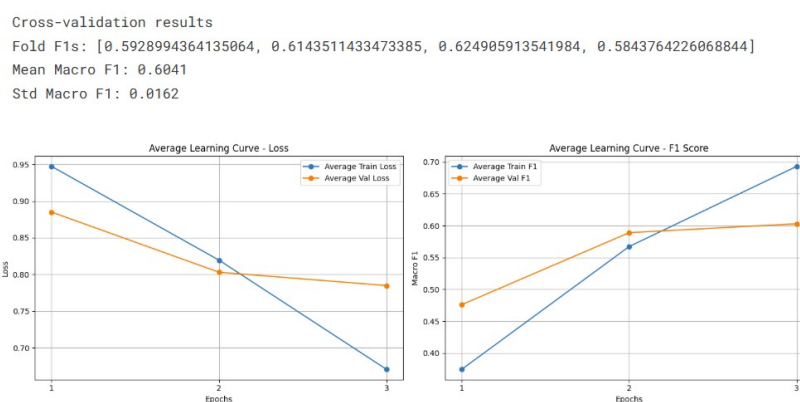


Figure 29:

The loss curves for this experiment follow a similar pattern to those observed in the

previous experiments, with the training loss decreasing steadily across epochs and the validation loss remaining relatively stable.

6.0.11. Overall Discussion. Overall, the BERT experiments showed that the baseline configuration was a strong and stable starting point, but it was not the best-performing setup in terms of mean macro F1-score. The baseline model, trained with batch size 8, learning rate 2×10^{-5} , and 3 epochs, achieved a mean macro F1-score of 0.6009 with a standard deviation of 0.0161.

Changing only the batch size or learning rate did not improve performance. Reducing the batch size to 4 achieved a macro F1-score of 0.5958, while increasing the learning rate to 3×10^{-5} reduced the macro F1-score to 0.5869 and increased instability across folds. Using a larger batch size of 16 together with the higher learning rate further reduced performance, achieving a macro F1-score of 0.5815.

The weighted-loss experiments showed that class weighting can improve the model's class-level behavior, although the first two configurations did not surpass the baseline. The configurations with weights [1.6, 1.0, 1.5] and [1.4, 1.0, 1.5] achieved macro F1-scores of 0.5940 and 0.5964, respectively. These results suggest that weighted loss helped the model pay more attention to the minority classes, but the choice of weights was important.

The striding-window input representation did not improve performance, achieving a macro F1-score of 0.5835. Similarly, increasing the number of training epochs to 4 did not lead to better generalization. Both the non-weighted and weighted 4-epoch experiments produced results close to the baseline, but the learning curves suggested early signs of overfitting.

The best overall result was obtained with Experiment 10, which combined class weights [1.3, 1.0, 1.5] with a warm-up learning rate scheduler. This configuration achieved a mean macro F1-score of 0.6041, the highest among all tested BERT setups. Therefore, Experiment 10 was selected as the final BERT configuration.

6.1. DistilBERT

In this section, we present the experiments conducted using the `distilbert-base-uncased` model. DistilBERT was tested as a lighter and more computationally efficient alternative to BERT. The goal of these experiments was to evaluate whether a smaller Transformer model could achieve competitive performance on the clarity classification task while requiring fewer computational resources.

As in the BERT experiments, the model was evaluated using 4-fold cross-validation, and the main metric used for comparison was the macro F1-score. This metric was selected because the dataset is imbalanced and macro F1-score provides a more balanced evaluation across all classes.

6.1.1. Experiment 1: Baseline DistilBERT Model. In the first experiment, we trained a baseline DistilBERT model using a standard configuration. The model was initialized from the `distilbert-base-uncased` checkpoint and was trained using the paired question-answer input format. The training was performed for 3 epochs, with a batch size of 8 and a learning rate of 2×10^{-5} . Standard cross-entropy loss was used, without class weighting or additional input-processing techniques.


```

=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.5772 ± 0.0115

Fold execution times:
Fold 1: 4.56 minutes
Fold 2: 4.68 minutes
Fold 3: 4.70 minutes
Fold 4: 4.70 minutes

Total time for all 4 folds: 18.65 minutes
Average time per fold: 4.66 minutes

Overall Classification Report (Out-Of-Fold Predictions)

```

	precision	recall	f1-score	support
0	0.63	0.33	0.43	1041
1	0.67	0.86	0.76	2009
2	0.60	0.50	0.55	352
accuracy			0.66	3402
macro avg	0.63	0.56	0.58	3402
weighted avg	0.65	0.66	0.63	3402

Figure 30:

The baseline model achieved a mean Macro F1-score of 0.5772 with a standard deviation of 0.0115, while the weighted F1-score was 0.63. These results were lower than the corresponding baseline BERT results, indicating that DistilBERT was less effective in this initial configuration. Moreover, the difference between the weighted F1-score and the Macro F1-score suggests that the model performed better on the majority class than on the other classes. This was also reflected in the recall values, where the majority class achieved a high recall of 86%. However, the recall values for the remaining classes were considerably lower, showing that the model struggled to correctly identify a substantial portion of the non-dominant class samples. Therefore, although the model achieved reasonable overall performance, it had difficulty recognizing all classes equally well. This experiment served as the reference point for the following DistilBERT experiments.

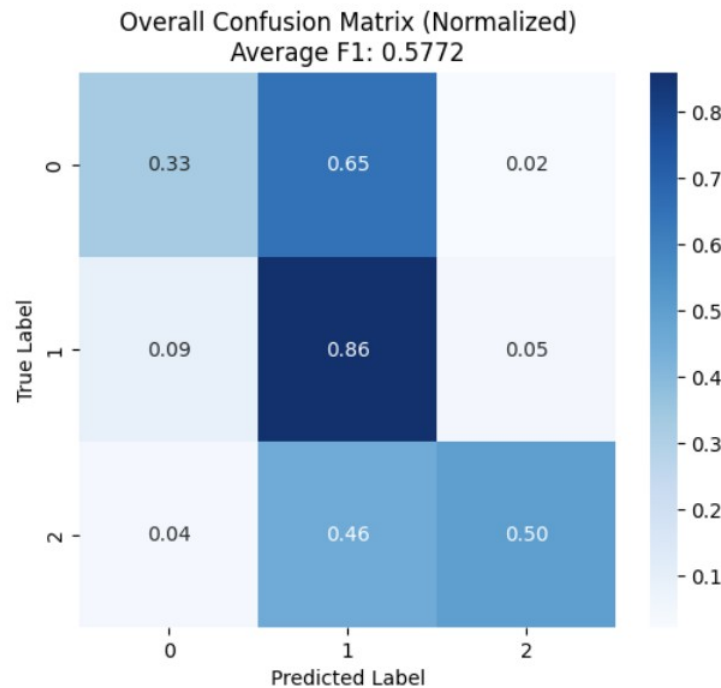


Figure 31:

Cross-validation results
 Fold F1s: [0.588806753316377, 0.5673576218931443, 0.5883097093569722, 0.564135018421062]
 Mean Macro F1: 0.5772
 Std Macro F1: 0.0115

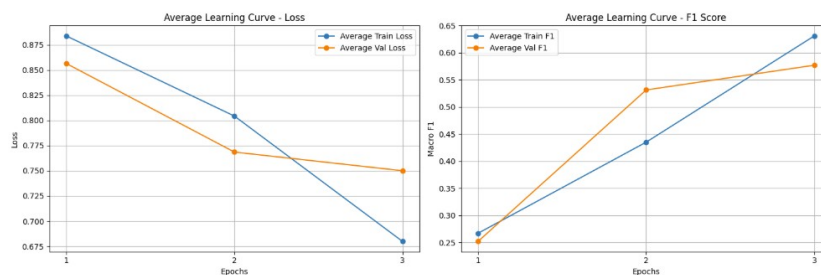


Figure 32:

The learning curves did not show clear signs of overfitting, as the validation loss remained relatively stable and did not diverge substantially from the training loss.

6.1.2. Experiment 2: Increased Learning Rate. In the second experiment, the learning rate was increased from 2×10^{-5} to 3×10^{-5} , while the batch size remained 8 and the number of epochs remained 3. The purpose of this experiment was to examine whether a slightly larger learning rate could improve convergence and validation performance.

This configuration achieved a mean macro F1-score of 0.5853 with a standard deviation of 0.0199. The validation accuracy remained approximately 0.66 and the weighted F1-score increased to 0.64. Compared to the baseline, this setup provided a small im-

provement in macro F1-score. However, the higher standard deviation indicates that the model became slightly less stable across the folds.

6.1.3. Experiment 3: Smaller Batch Size. In the third experiment, the batch size was reduced from 8 to 4, while the learning rate was kept at 2×10^{-5} and the number of epochs remained 3. The goal was to test whether smaller batches could improve generalization by introducing more frequent parameter updates.

The model achieved a mean macro F1-score of 0.5832 with a standard deviation of 0.0261. The validation accuracy was approximately 0.65 and the weighted F1-score was 0.64. This was slightly better than the baseline, but worse than the configuration with the increased learning rate. The higher standard deviation also suggests that the smaller batch size made training less stable across folds.

6.1.4. Experiment 4: Smaller Batch Size with Increased Learning Rate. In the fourth experiment, both the batch size and the learning rate were modified. The batch size was set to 4 and the learning rate was increased to 3×10^{-5} . The purpose of this experiment was to examine whether combining more frequent updates with a larger learning rate could improve the model's performance.

```
=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.5933 ± 0.0177

Fold execution times:
Fold 1: 5.85 minutes
Fold 2: 5.84 minutes
Fold 3: 5.84 minutes
Fold 4: 5.85 minutes

Total time for all 4 folds: 23.39 minutes
Average time per fold: 5.85 minutes

Overall Classification Report (Out-Of-Fold Predictions)

```

	precision	recall	f1-score	support
0	0.63	0.37	0.46	1041
1	0.69	0.85	0.76	2009
2	0.61	0.52	0.56	352
accuracy			0.67	3402
macro avg	0.64	0.58	0.59	3402
weighted avg	0.66	0.67	0.65	3402

Figure 33:

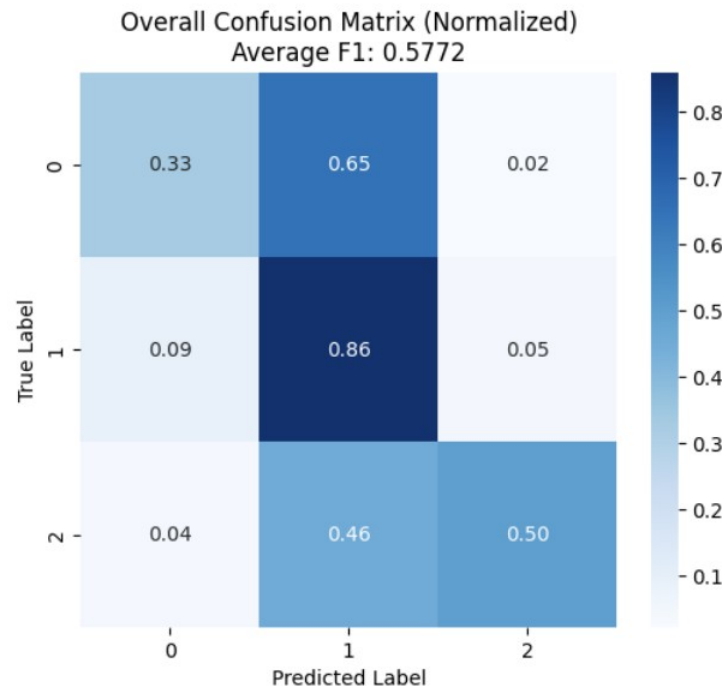


Figure 34:

This configuration achieved a mean Macro F1-score of 0.5933 with a standard deviation of 0.0177. It also achieved an accuracy of approximately 0.67 and a weighted F1-score of 0.65. Among the learning-rate and batch-size experiments, this was the best configuration. It outperformed the baseline and showed that the combination of a smaller batch size and a higher learning rate helped DistilBERT generalize better in this setup. However, the classification report and confusion matrix still indicate the same underlying difficulty in separating the classes. Although overall performance improved, the model continued to perform better on the majority class, while the non-dominant classes remained harder to distinguish.

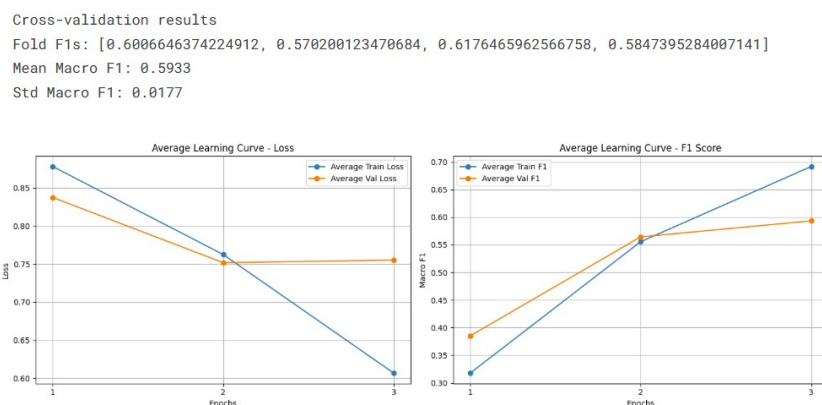


Figure 35:

The learning curves did not show clear signs of overfitting, as the validation loss remained relatively stable and did not diverge substantially from the training loss.

6.1.5. Experiment 5: Larger Batch Size. In the fifth experiment, the batch size was increased to 16, while the learning rate was set to 3×10^{-5} and the model was trained for 3 epochs. This experiment was designed to test whether larger batches could produce more stable gradient estimates and improve training efficiency.

```
=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.5657 ± 0.0368

Fold execution times:
Fold 1: 4.12 minutes
Fold 2: 4.14 minutes
Fold 3: 4.14 minutes
Fold 4: 4.14 minutes

Total time for all 4 folds: 16.54 minutes
Average time per fold: 4.13 minutes

Overall Classification Report (Out-Of-Fold Predictions)
      precision    recall  f1-score   support

0         0.61      0.33      0.42      1041
1         0.67      0.86      0.75      2009
2         0.61      0.47      0.53       352

 accuracy          0.66      3402
 macro avg         0.63      0.55      0.57      3402
weighted avg         0.65      0.66      0.63      3402
```

Figure 36:

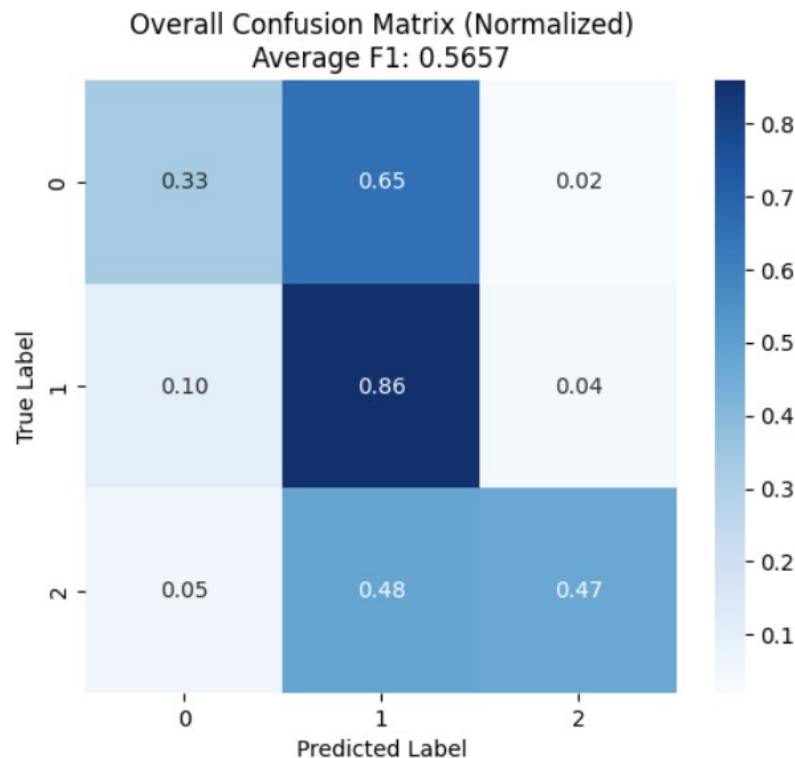


Figure 37:

The model achieved a mean Macro F1-score of 0.5657 with a standard deviation of 0.0368. The validation accuracy was approximately 0.66 and the weighted F1-score was 0.63. This result was lower than the baseline and also had the highest standard deviation among the DistilBERT experiments. Therefore, larger batches were not suitable for this model and task, possibly because they reduced the number of parameter updates per epoch and limited the model's ability to adapt well to the training data. In addition, the classification report and confusion matrix showed that the model performed even worse in separating the classes, with stronger bias toward the majority class and poorer recognition of the non-dominant classes.

6.1.6. Experiment 6: Weighted Cross-Entropy Loss. In the sixth experiment, weighted cross-entropy loss was used to address class imbalance. The selected class weights were $[1.4, 1.0, 1.5]$, assigning higher importance to the underrepresented classes. This experiment used the best hyperparameter setup from the previous experiments, with batch size 4 and learning rate 3×10^{-5} . The model was trained for 3 epochs.

This configuration achieved a mean macro F1-score of 0.5890 with a standard deviation of 0.0208. The validation accuracy was approximately 0.65 and the weighted F1-score was 0.63. Although this result was higher than the baseline, it did not outperform Experiment 4, which achieved a macro F1-score of 0.5933 without weighted loss. This suggests that weighted loss slightly improved minority-class sensitivity, but did not provide a clear overall improvement in this configuration.

6.1.7. Experiment 7: Striding-Window Input Representation. In the seventh experiment, a striding-window input representation was tested. This method was intro-

duced to better handle longer question–answer pairs that may exceed the maximum input length of the model. The maximum sequence length was set to 512 tokens and the stride was set to 128 tokens. The model was trained for 3 epochs, with batch size 8 and learning rate 2×10^{-5} .

The model achieved a mean macro F1-score of 0.5619 with a standard deviation of 0.0100. The validation accuracy was approximately 0.64 and the weighted F1-score was 0.62. This was lower than both the baseline and the best tuned configuration from Experiment 4. Although the striding-window approach allowed the model to process longer inputs, the additional context did not improve classification performance. This indicates that the most useful information was already contained in the initial part of the paired input, while the additional overlapping segments introduced noise or made the prediction process more complex.

6.1.8. Experiment 8: More Training Epochs. In the eighth experiment, the number of epochs was increased from 3 to 5. This experiment extended the best non-weighted configuration from Experiment 4, using batch size 4 and learning rate 3×10^{-5} . The goal was to examine whether longer training could improve validation performance.

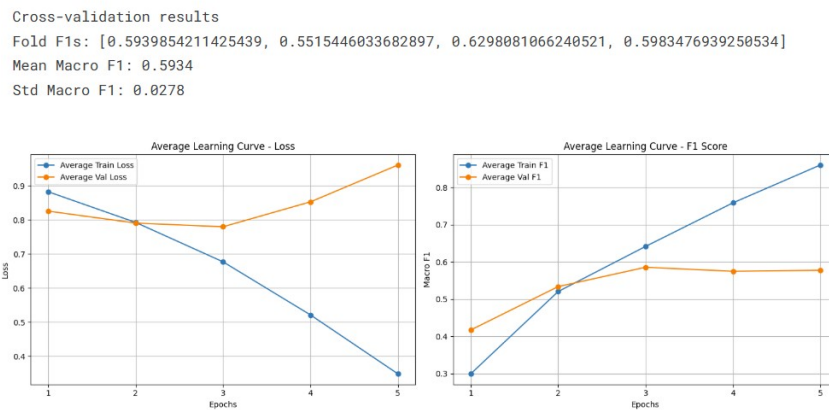


Figure 38:

This experiment achieved a mean Macro F1-score of 0.5934 with a standard deviation of 0.0278. This result was almost identical to Experiment 4, which achieved a Macro F1-score of 0.5933. However, the standard deviation increased, indicating less stable performance across folds. The learning curves suggest that longer training did not provide a clear generalization benefit and led to mild overfitting after the third epoch. More specifically, after epoch 3, the training loss continued to decrease, while the validation loss started to increase. At the same time, the training F1-score continued to improve, whereas the validation F1-score remained almost stable. This indicates that the model started to fit the training data more strongly without achieving better validation performance. From the Macro F1 learning curve, it can be observed that after the third epoch the validation Macro F1-score reaches a plateau, indicating that additional training does not lead to further improvement in generalization performance.

6.1.9. Experiment 9: More Epochs with Weighted Loss. In the final DistilBERT experiment, the model was trained for 4 epochs using weighted cross-entropy loss. The class

weights were set to $[1.3, 1.0, 1.5]$, while the batch size remained 4 and the learning rate remained 3×10^{-5} . This setup combined the best hyperparameter configuration with a less aggressive weighted-loss strategy.

This experiment achieved the best overall DistilBERT result, reaching a mean macro F1-score of 0.5940 with a standard deviation of 0.0137. Compared to Experiment 4, the improvement in macro F1-score was very small, but the lower standard deviation suggests that this configuration was more stable.

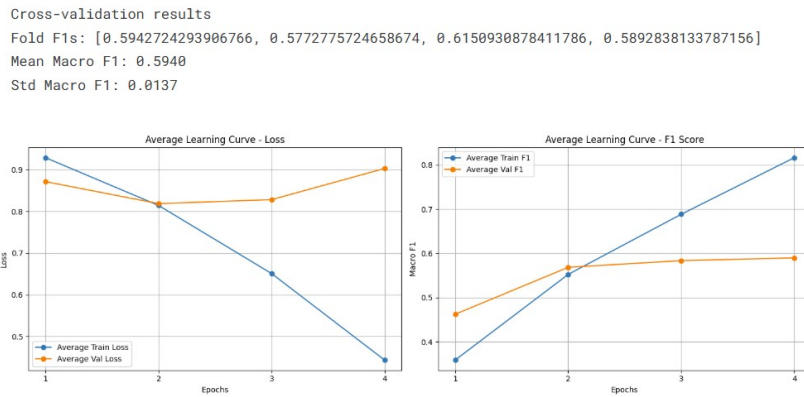


Figure 39:

Therefore, weighted loss combined with 4 training epochs provided a slight benefit, mainly in terms of balanced class recognition and stability. However, the learning curves indicate that signs of overfitting appear after the third epoch, as the training performance continues to improve while the validation performance does not show a corresponding improvement. Therefore, the better result of this configuration can mainly be attributed to the use of class weights, rather than to the additional training epoch. From the Macro F1 learning curve, it can be observed that after the third epoch the validation Macro F1-score reaches a plateau, indicating that additional training does not lead to further improvement in generalization performance.

6.1.10. Overall Discussion. Overall, the DistilBERT experiments showed that the baseline configuration was not the best-performing setup. The baseline model achieved a mean macro F1-score of 0.5772, while several tuning experiments improved performance. Increasing the learning rate to 3×10^{-5} improved the score to 0.5853, and reducing the batch size to 4 achieved 0.5832. The strongest non-weighted configuration was obtained by combining both changes, using batch size 4 and learning rate 3×10^{-5} , which achieved a mean macro F1-score of 0.5933.

The larger batch-size experiment did not improve performance and achieved the lowest macro F1-score of 0.5657. Similarly, the striding-window input representation performed worse than the baseline, with a macro F1-score of 0.5619. These results suggest that increasing either batch size or input complexity was not beneficial for DistilBERT in this task.

The weighted-loss experiments produced small but meaningful changes in the model's class-level behavior. The first weighted-loss setup achieved a macro F1-score of 0.5890, which was higher than the baseline but lower than the best non-weighted configuration. The best overall result was obtained in Experiment 9, where the model was

trained for 4 epochs with class weights [1.3, 1.0, 1.5]. This configuration achieved a mean macro F1-score of 0.5940 with a standard deviation of 0.0137.

Therefore, the strongest DistilBERT configuration was Experiment 9, using batch size 4, learning rate 3×10^{-5} , 4 training epochs, and weighted cross-entropy loss with class weights [1.3, 1.0, 1.5]. However, the improvement over Experiment 4 was marginal. For this reason, Experiment 9 can be considered the best-performing configuration, while Experiment 4 remains an almost equally strong and simpler alternative.

6.2. DeBERTa Model

In this section, we present the experiments conducted using the `microsoft/deberta-v3-base` model. DeBERTa was tested as a stronger Transformer-based alternative to BERT and DistilBERT for the clarity classification task. The goal was to examine whether this architecture could improve classification performance, especially for the minority classes.

Unlike BERT and DistilBERT, DeBERTa showed significant instability during fine-tuning. The baseline configuration produced NaN losses from the first epoch, and therefore the following experiments focused mainly on stabilizing the training process. For this reason, the DeBERTa experiments were treated as exploratory tuning experiments rather than an exhaustive hyperparameter search. Due to the higher computational cost of DeBERTa, most stabilized experiments were evaluated using 2-fold stratified cross-validation.

6.2.1. Experiment 1: Baseline DeBERTa Model. In the first experiment, DeBERTa was fine-tuned using a baseline configuration similar to the initial BERT setup. The model was initialized from the `microsoft/deberta-v3-base` checkpoint and trained using paired question–answer inputs with a maximum sequence length of 512 tokens. The model was trained for 3 epochs, using a batch size of 8 and a learning rate of 2×10^{-5} .

```
=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.1562 ± 0.0002

Fold execution times:
Fold 1: 5.51 minutes
Fold 2: 5.42 minutes
Fold 3: 5.44 minutes
Fold 4: 5.43 minutes

Total time for all 4 folds: 21.80 minutes
Average time per fold: 5.45 minutes

Overall Classification Report (Out-Of-Fold Predictions):
```

	precision	recall	f1-score	support
0	0.31	1.00	0.47	1041
1	0.00	0.00	0.00	2009
2	0.00	0.00	0.00	352
accuracy			0.31	3402
macro avg	0.10	0.33	0.16	3402
weighted avg	0.09	0.31	0.14	3402

Figure 40:

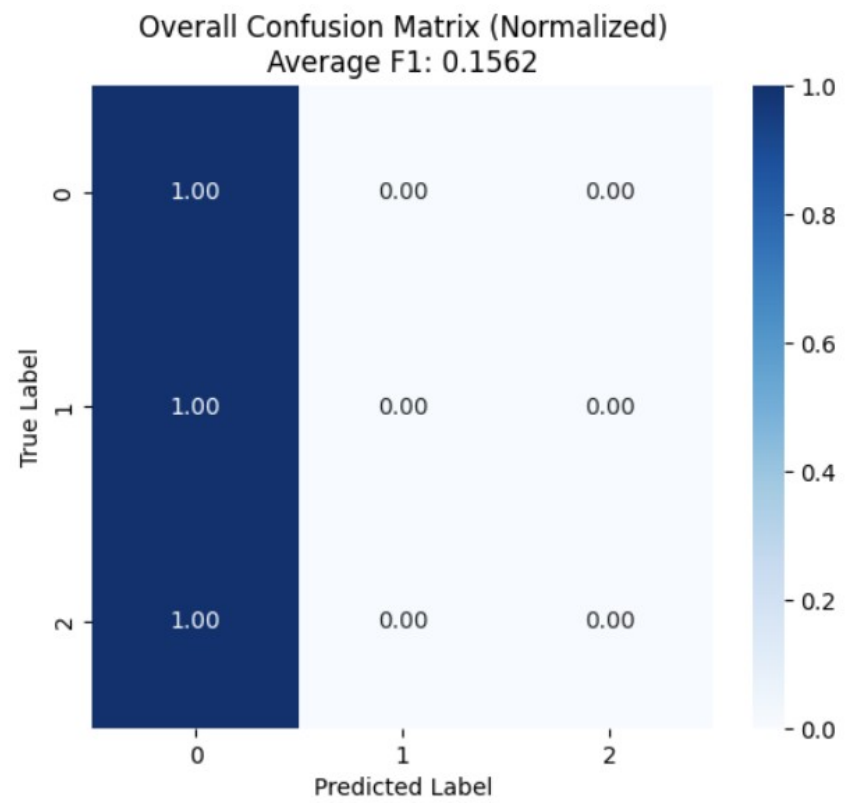


Figure 41:

This configuration was not successful. Both the training loss and validation loss became NaN from the first epoch, indicating numerical instability during fine-tuning. The model achieved a very low mean macro F1-score of 0.1562 with a standard deviation of 0.0002 across the 4 folds. The validation accuracy was approximately 0.31, while the weighted F1-score was 0.14.

The classification report and the confusion matrix showed that the model predicted all validation samples as a single class. As a result, it failed to recognize two of the three classes, which received F1-scores of 0.00. This shows that the baseline DeBERTa configuration did not learn useful decision boundaries between the clarity labels. Therefore, additional stabilization was necessary.

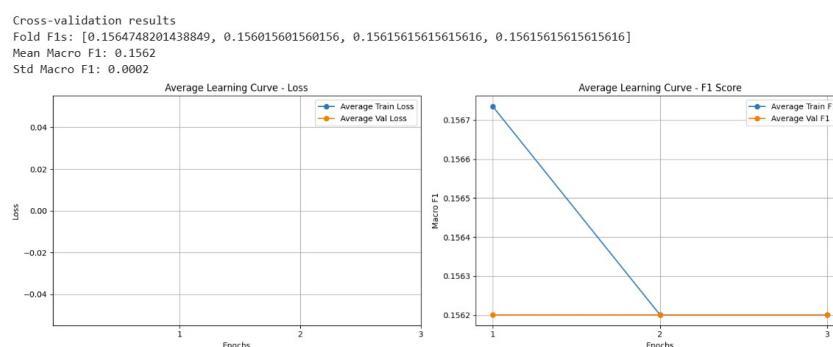


Figure 42:

As shown in the learning curves, the loss plot does not display a meaningful curve

because both the training and validation losses became NaN from the first epoch. Therefore, the loss values could not be properly visualized or interpreted. In contrast, the Macro F1-score appears as an flat line across epochs, indicating that the model did not improve during training. This further confirms that the baseline DeBERTa configuration failed to learn useful patterns from the data and suffered from severe numerical instability.

6.2.2. Experiment 2: Lower Learning Rate and Float32 Precision. In the second experiment, the learning rate was reduced from 2×10^{-5} to 1×10^{-5} in order to make the optimization process more stable. In addition, the model was explicitly trained using float32 precision, which provides more stable numerical computations. The model was trained for 3 epochs with a batch size of 8. Unlike the baseline experiment, this setup used 2-fold stratified cross-validation to reduce computational cost while testing whether the stabilization changes could solve the NaN loss problem.

```
=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.3883 ± 0.0229

Fold execution times:
Fold 1: 11.26 minutes
Fold 2: 11.26 minutes

Total time for all 2 folds: 22.52 minutes
Average time per fold: 11.26 minutes

Overall Classification Report (Out-Of-Fold Predictions):
      precision    recall  f1-score   support

0         0.55        0.02        0.04       1041
1         0.61        0.96        0.75       2009
2         0.50        0.31        0.38        352

accuracy          0.60       3402
macro avg         0.55        0.43        0.39       3402
weighted avg      0.58        0.60        0.49       3402
```

Figure 43:

These changes successfully prevented the NaN losses. The model completed training with finite training and validation losses, showing that reducing the learning rate and using float32 precision improved the numerical stability of DeBERTa fine-tuning.

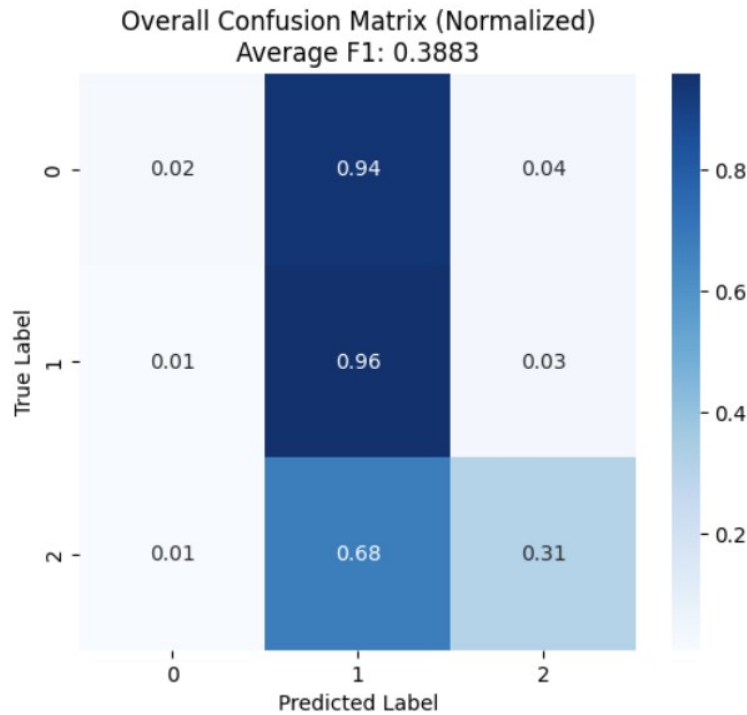


Figure 44:

The model achieved a mean macro F1-score of 0.3883 with a standard deviation of 0.0229. The validation accuracy was approximately 0.60, while the weighted F1-score was 0.49. Although this was a clear improvement over the unstable baseline, the performance was still much lower than BERT and DistilBERT. The classification report and confusion matrix showed that the model still relied heavily on the Ambivalent class. The only exception was the Clear Non-Reply class, where 31% of the samples were correctly identified. However, the remaining classes were largely misclassified as Ambivalent, showing that the model still struggled to learn clear decision boundaries between the three clarity labels.

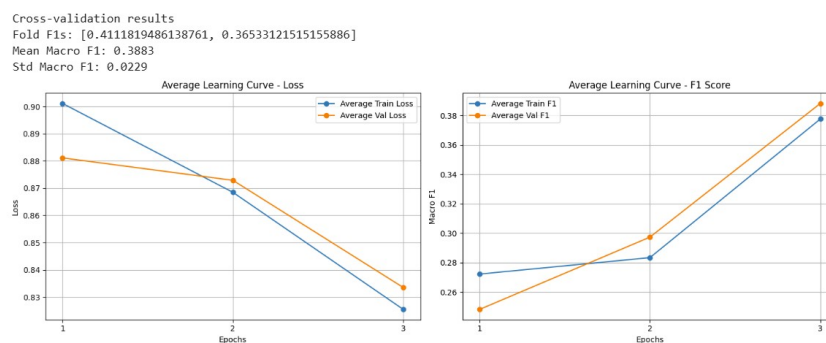


Figure 45:

No clear signs of overfitting are observed in this experiment, since the validation loss decreases together with the training loss, while the validation Macro F1-score continues to increase.

6.2.3. Experiment 3: Weighted Loss and Warm-Up Scheduler. After the NaN loss issue was resolved, the next goal was to improve class separation. In Experiment 2, the model remained biased toward one class, so weighted cross-entropy loss was introduced to give more importance to the underrepresented classes. A warm-up scheduler was also added to make the beginning of training more gradual.

The class weights were set to $[1.3, 1.0, 1.5]$, the learning rate was reduced further to 5×10^{-6} , and the batch size remained 8. The model was trained for 3 epochs using 2-fold stratified cross-validation.

```
=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.3096 ± 0.0389

Fold execution times:
Fold 1: 11.10 minutes
Fold 2: 11.13 minutes

Total time for all 2 folds: 22.23 minutes
Average time per fold: 11.11 minutes

Overall Classification Report (Out-Of-Fold Predictions):
      precision    recall  f1-score   support

0         0.45        0.01        0.02       1041
1         0.60        0.97        0.74       2009
2         0.36        0.12        0.18        352

 accuracy         0.59         3402
 macro avg         0.47         0.37         0.32         3402
 weighted avg         0.53         0.59         0.47         3402
```

Figure 46:

The training process remained stable and did not produce NaN losses. However, the classification performance decreased compared to Experiment 2. The model achieved a mean macro F1-score of 0.3096 with a standard deviation of 0.0389. The validation accuracy was approximately 0.59 and the weighted F1-score was 0.47.

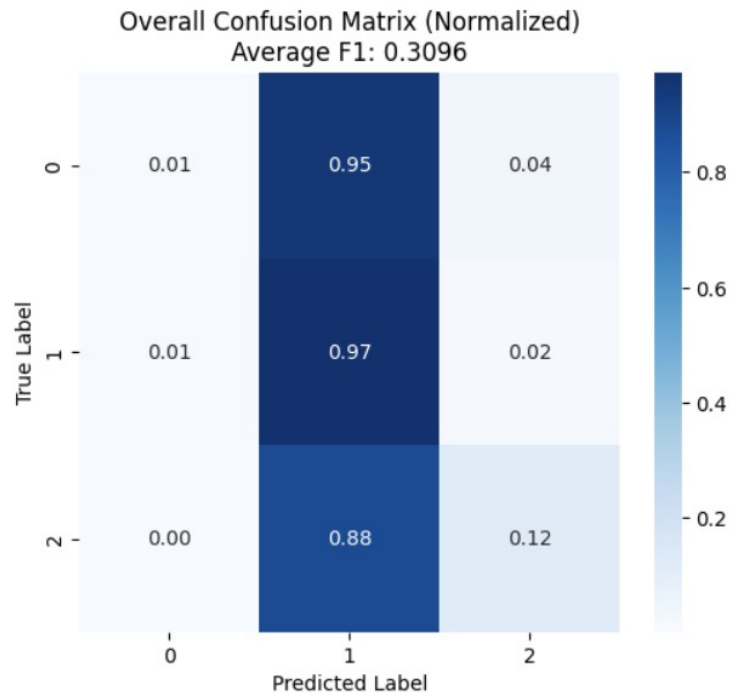


Figure 47:

Although weighted loss and warm-up scheduling were introduced to improve minority-class recognition, they did not improve performance in this configuration. A likely explanation is that the very low learning rate slowed down convergence. The training logs showed that the loss was still decreasing by the third epoch, suggesting that the model may have required more training time.

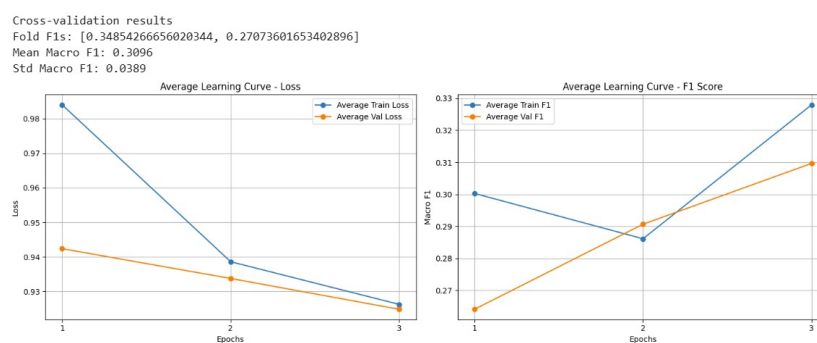


Figure 48:

No clear signs of overfitting are observed in this experiment, since the validation loss decreases together with the training loss, while the validation Macro F1-score continues to increase.

6.2.4. Experiment 4: Stronger Stabilization and Multi-Factor Tuning. In the fourth experiment, a stronger tuning strategy was tested. The previous experiments showed that DeBERTa could be stabilized with a lower learning rate and float32 precision, but

it still struggled to separate the three classes effectively. Therefore, several changes were applied together.

```
=====
Overall Cross Validation Results
=====

Average Validation F1 Score: 0.5028 ± 0.0335

Fold execution times:
Fold 1: 29.92 minutes
Fold 2: 29.84 minutes

Total time for all 2 folds: 59.77 minutes
Average time per fold: 29.88 minutes

Overall Classification Report (Out-Of-Fold Predictions):
```

	precision	recall	f1-score	support
0	0.43	0.63	0.51	1041
1	0.69	0.57	0.62	2009
2	0.55	0.30	0.38	352
accuracy			0.56	3402
macro avg	0.55	0.50	0.51	3402
weighted avg	0.59	0.56	0.56	3402

Figure 49:

The number of training epochs was increased to 8, allowing the model more time to learn with the reduced learning rate. Stronger class weights [1.8, 1.0, 1.5] were used to increase the importance of the underrepresented classes. The learning rate remained low at 5×10^{-6} , and the warm-up scheduler was kept. Gradient clipping was also added with a maximum gradient norm of 0.5 in order to limit large gradient updates and improve training stability.

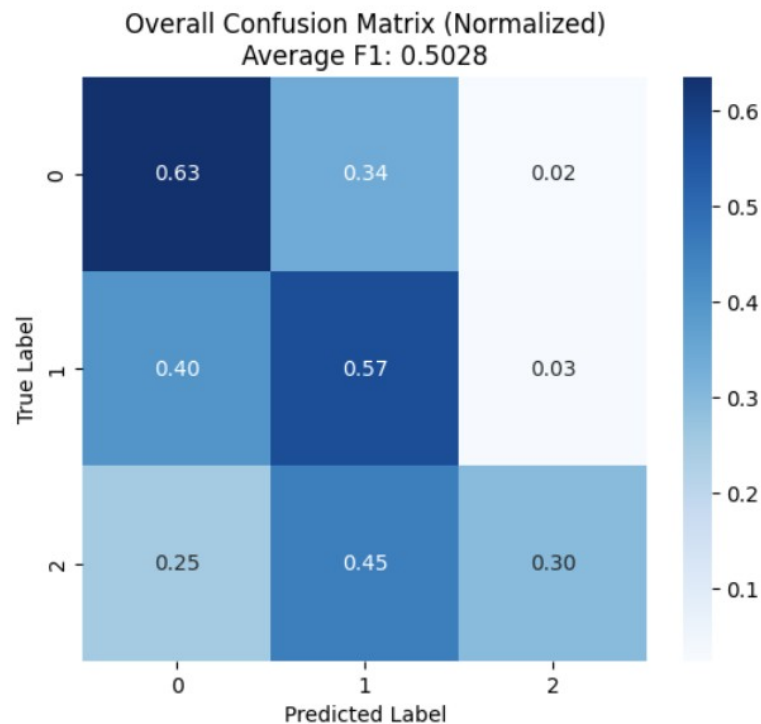


Figure 50:

This configuration produced the best DeBERTa result among the tested setups. The model achieved a mean Macro F1-score of 0.5028 with a standard deviation of 0.0335 across the 2 folds. This was a clear improvement over the previous DeBERTa experiments, especially Experiment 3, which achieved a Macro F1-score of 0.3096.

The results indicate that the stronger class weights had a substantial effect on the model's behavior. In particular, the model improved its ability to identify the Clear Reply class. However, this improvement came with a clear trade-off, as the performance on the majority Ambivalent class decreased considerably. This suggests that the stronger weighting shifted the model's attention toward the underrepresented classes, improving minority-class recognition but reducing its ability to correctly classify the dominant class.

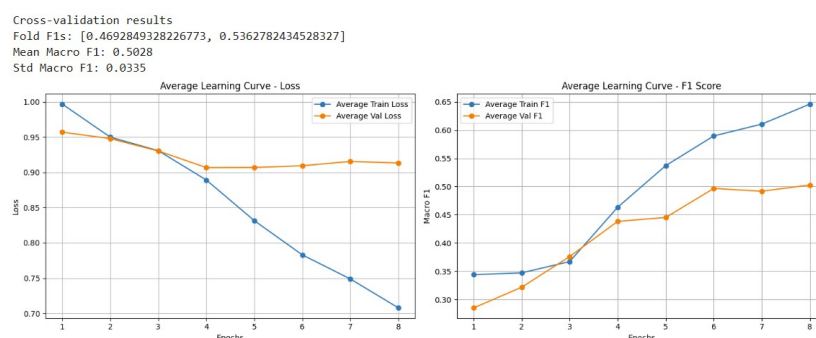


Figure 51:

The learning curves showed signs of mild overfitting after the middle epochs. The

training loss continued to decrease and the training macro F1-score continued to increase, while the validation loss stopped improving after around epoch 4. However, the validation macro F1-score continued to improve overall and reached its best value at epoch 8. Therefore, training until epoch 8 was justified based on the main evaluation metric, although future experiments should use early stopping to avoid unnecessary overfitting.

6.2.5. Overall Discussion. Overall, DeBERTa-v3-base was more difficult and more time-consuming to fine-tune than BERT and DistilBERT. The baseline experiment was numerically unstable and produced NaN losses from the first epoch, resulting in a very low macro F1-score of 0.1562. Reducing the learning rate and using float32 precision solved the NaN loss problem, but the model still struggled to separate the three clarity classes effectively.

The best DeBERTa result was obtained in Experiment 4, where multiple stabilization techniques were combined: a very low learning rate, float32 precision, stronger class weights, 8 training epochs, warm-up scheduling, and gradient clipping. This setup achieved a mean macro F1-score of 0.5028 with a standard deviation of 0.0335 across the 2 folds. Although this was a clear improvement over the previous DeBERTa configurations, it was still lower than the best BERT and DistilBERT results.

For this reason, DeBERTa was not selected as the final model. Although it is a strong pretrained architecture, it required substantially more tuning time and computational resources while achieving lower performance than the simpler BERT-based models. Within the scope of this work, DeBERTa was therefore less efficient and less effective than the selected BERT configuration.

Table 2: Summary of BERT fine-tuning experiments and validation results.

Exp.	Description	Folds	Epochs	Batch	Learning Rate / Extra Setting	Mean Macro F1	Std	Acc.	Macro P	Macro R	Weighted F1
1	Baseline BERT model.	4	3	8	2×10^{-5}	0.6009	0.0161	0.66	0.62	0.59	0.65
2	Batch-size experiment with smaller batch size.	4	3	4	2×10^{-5}	0.5958	0.0167	0.66	0.62	0.59	0.65
3	Learning-rate experiment with increased learning rate.	4	3	8	3×10^{-5}	0.5869	0.0341	0.66	0.63	0.58	0.64
4	Larger batch-size experiment.	4	3	16	3×10^{-5}	0.5815	0.0272	0.65	0.62	0.58	0.63
5	Weighted loss experiment to handle class imbalance.	4	3	8	2×10^{-5} , class weights [1.6, 1.0, 1.5]	0.5940	0.0069	0.64	0.60	0.60	0.63
6	Second weighted loss experiment with adjusted class weights.	4	3	8	2×10^{-5} , class weights [1.4, 1.0, 1.5]	0.5964	0.0080	0.65	0.61	0.59	0.64
7	Input representation experiment using a striding-window approach for long sequences.	4	3	8	2×10^{-5} , max length 512, stride 128	0.5835	0.0169	0.65	0.62	0.58	0.63
8	Epoch experiment using more training epochs.	4	4	8	2×10^{-5}	0.5983	0.0216	0.66	0.64	0.58	0.65
9	Weighted loss experiment using more training epochs.	4	4	8	2×10^{-5} , class weights [1.3, 1.0, 1.5]	0.5981	0.0063	0.65	0.61	0.59	0.64
10	Weighted loss experiment with warm-up scheduler.	4	3	8	2×10^{-5} , class weights [1.3, 1.0, 1.5], warm-up	0.6041	0.0162	0.65	0.62	0.59	0.64

Table 3: Summary of DistilBERT fine-tuning experiments and validation results.

Exp.	Description	Folds	Epochs	Batch	Learning Rate / Extra Setting	Mean Macro F1	Std	Acc.	Macro P	Macro R	Weighted F1
1	Baseline DistilBERT model using standard cross-entropy loss.	4	3	8	2×10^{-5}	0.5772	0.0115	0.66	0.63	0.56	0.63
2	Learning-rate experiment with increased learning rate.	4	3	8	3×10^{-5}	0.5853	0.0199	0.66	0.63	0.57	0.64
3	Batch-size experiment with smaller batch size.	4	3	4	2×10^{-5}	0.5832	0.0261	0.65	0.62	0.57	0.64
4	Combined smaller batch size and increased learning rate.	4	3	4	3×10^{-5}	0.5933	0.0177	0.67	0.64	0.58	0.65
5	Larger batch-size experiment.	4	3	16	3×10^{-5}	0.5657	0.0368	0.66	0.63	0.55	0.63
6	Weighted cross-entropy loss experiment for class imbalance.	4	3	4	3×10^{-5} , class weights [1.4, 1.0, 1.5]	0.5890	0.0208	0.65	0.62	0.57	0.63
7	Input representation experiment using striding-window approach.	4	3	8	2×10^{-5} , max length 512, stride 128	0.5619	0.0100	0.64	0.59	0.55	0.62
8	Epoch experiment using more training epochs.	4	5	4	3×10^{-5}	0.5934	0.0278	0.64	0.61	0.58	0.64
9	Weighted loss experiment using more training epochs.	4	4	4	3×10^{-5} , class weights [1.3, 1.0, 1.5]	0.5940	0.0137	0.64	0.61	0.58	0.64

Table 4: Summary of DeBERTa fine-tuning experiments and validation results.

Exp.	Description	Folds	Epochs	Batch	Learning Rate / Extra	Set-Mean Macro F1	Std	Acc.	Macro P	Macro R	Weighted F1
					ting						
1	Baseline DeBERTa model using standard cross-entropy loss.	4	3	8	2×10^{-5}	0.1562	0.0002	0.31	0.10	0.33	0.14
2	Lower learning rate and float32 precision to fix NaN losses.	2	3	8	1×10^{-5} , float32	0.3883	0.0229	0.60	0.55	0.44	0.49
3	Weighted loss with warm-up scheduler and lower learning rate.	2	3	8	5×10^{-6} , class weights [1.3, 1.0, 1.5], warm-up	0.3096	0.0389	0.59	0.47	0.37	0.47
4	Multi-factor tuning with stronger class weights and gradient clipping.	2	8	8	5×10^{-6} , class weights [1.8, 1.0, 1.5], warm-up, gradient clipping	0.5028	0.0335	0.56	0.50	0.51	0.56

6.3. Results

7. Conclusion and model selection

After evaluating BERT, DistilBERT, and DeBERTa, several differences were observed not only in terms of classification performance, but also in terms of training stability, computational cost, and practical usability.

First, DistilBERT was the fastest and least computationally demanding model. Since it is a distilled version of BERT, it contains fewer parameters and requires less memory and training time. This made it easier to experiment with different hyperparameter configurations, such as batch size, learning rate, number of epochs, and weighted loss. Although its best macro F1-score was slightly lower than BERT's, DistilBERT remained a strong and efficient alternative, especially in scenarios where computational resources or training time are limited.

BERT provided the best balance between performance, stability, and computational cost. It was more computationally demanding than DistilBERT, but it remained stable during fine-tuning and did not require extensive stabilization techniques. The baseline BERT configuration already achieved strong performance, and the best result was obtained after introducing class weighting and a warm-up scheduler. Overall, BERT achieved the highest macro F1-score among the three models, while remaining practical to train and tune. For this reason, it was selected as the final model.

DeBERTa was the most difficult model to fine-tune and the most computationally expensive when training time was compared at the fold level. Although it is a stronger pretrained architecture in general, it showed numerical instability in this task, with the baseline experiment producing NaN losses from the first epoch. Additional stabilization techniques were required, including a lower learning rate, float32 precision, warm-up scheduling, stronger class weights, more training epochs, and gradient clipping. Even after these adjustments, DeBERTa did not outperform BERT or DistilBERT. Its best result was clearly improved compared to its unstable baseline, but it remained below the best BERT-based configurations. Therefore, within the available time and computational resources, DeBERTa was less efficient and less effective for this task.

In terms of training time, the three models also showed clear differences. DistilBERT was the fastest and most efficient model, requiring approximately 24.6 minutes per experiment on average and around 6.1 minutes per fold. BERT required more time,

with an average of about 41.0 minutes per experiment and approximately 10.2 minutes per fold, but its training time remained manageable given its stronger performance. DeBERTa required about 31.6 minutes per experiment on average and around 12.6 minutes per fold. Although its average time per experiment was lower than BERT's in these runs, this was mainly because some DeBERTa experiments were evaluated with fewer folds. When comparing the models at the fold level, DeBERTa was the slowest model, which confirms that it was the most computationally expensive to fine-tune. Therefore, the timing results further support the overall conclusion: DistilBERT was the most efficient model, BERT provided the best balance between performance and cost, and DeBERTa was the most demanding model without offering better classification performance.

In terms of classification performance, BERT achieved the best overall result, followed by DistilBERT and then DeBERTa. The best BERT configuration achieved a mean macro F1-score of 0.6041, while the best DistilBERT configuration achieved 0.5940. The difference between these two models was relatively small, suggesting that DistilBERT was able to retain much of BERT's performance while being faster and lighter. In contrast, DeBERTa achieved a best macro F1-score of 0.5028, which was substantially lower than the other two models.

From a practical perspective, DistilBERT was the most efficient model, BERT was the best overall model, and DeBERTa was the most expensive and difficult to optimize. Therefore, if the goal is maximum predictive performance, BERT is the preferred choice. If the goal is faster experimentation or deployment with limited resources, DistilBERT is a strong alternative. DeBERTa may require more extensive hyperparameter tuning and computational resources before it can become competitive for this specific dataset.

Overall, the comparison shows that model complexity alone does not guarantee better performance. Although DeBERTa is architecturally more advanced, it was not the most effective model in this work. The simpler BERT-based models were more stable, easier to fine-tune, and achieved better macro F1-scores. As a result, the final selected model was BERT with weighted cross-entropy loss and a warm-up scheduler (Experiment 10 from Bert).

7.1. Comparison with the first project

In the first project, the same classification problem was approached using traditional text representation methods, mainly TF-IDF and Word2Vec, combined with a Logistic Regression classifier. These methods required more manual preprocessing and feature engineering compared to the Transformer-based approach used in the current project. Their performance depended strongly on preprocessing decisions, such as text cleaning, tokenization, stopword handling, and the choice of vector representation.

The most stable result in the first project was obtained with Word2Vec, which reached an F1-score of approximately 49% without clear signs of overfitting. However, this score was considerably lower than the results achieved by the Transformer-based models in the current project. This suggests that although Word2Vec was able to capture some semantic information, its static word representations were not sufficient to model the complexity of the clarity classification task.

On the other hand, the TF-IDF-based models were more problematic. Some configurations reached scores of around 60%, but these results were accompanied by strong

overfitting. This indicates that the models were likely learning dataset-specific lexical patterns rather than generalizable features. Therefore, although TF-IDF sometimes produced higher apparent scores, it was not considered a reliable solution for the task.

The current Transformer-based models achieved better and more stable results. Even the baseline BERT model, without extensive additional tuning, achieved a mean Macro F1-score of 0.6009 across 4-fold cross-validation. This result was higher than the stable Word2Vec baseline and was obtained using a more robust evaluation procedure.

The improvement can be explained by the nature of Transformer-based representations. Unlike TF-IDF, which relies mainly on word-frequency patterns, and Word2Vec, which assigns static vectors to words, BERT-based models produce contextual representations. This means that the representation of each token depends on the surrounding text and on the relationship between the question and the answer. This is especially important for the clarity classification task, because the label often depends not only on the words used in the answer, but also on whether the answer directly addresses the specific question.

Moreover, the results from the first project suggest that the classes are not easily separable using specific keywords or simple lexical patterns. There do not appear to be specific words or topics that clearly distinguish one clarity class from another. As a result, traditional methods such as TF-IDF struggle to generalize, since they mainly rely on surface-level word occurrence. In contrast, Transformer-based models are better suited to this task because they can capture contextual meaning, sentence structure, and the semantic relationship between the question and the response.

Overall, the comparison with the first project shows that Transformer-based fine-tuning is more suitable for this problem than traditional vectorization methods. Word2Vec provided a more stable traditional baseline but remained limited in performance, while TF-IDF was more prone to overfitting despite sometimes achieving higher apparent scores. In contrast, BERT and DistilBERT achieved stronger and more stable Macro F1-scores, required less manual feature engineering, and were better able to model the semantic relationship between questions and answers.